FieryPath Specification

Fiery DFE-Press Control Path Interface





Revision History

Version	Change Log	Date	Author
0.1	Initial Draft	03/03/2016	Scott Crosby
0.2	Added Sequence Diagrams	03/30/2016	Aloke Bordia
0.3	Added Resource Actions/Commands	04/21/2016	Scott Crosby
0.4	Updated Sequences, WebSocket	04/28/2016	Aloke Bordia
0.5	Removed duplication between this and doc shared	05/13/2016	Naveen Pasumarthi
0.6	Split scenarios, updated events.	05/23/2016	Scott Crosby
0.7	Updates for SSL	06/06/2016	Aloke Bordia
0.8	Updates to REST URL formats	06/10/2016	Naveen Pasumarthi
0.9	Added Inks & Inputs Resource Action/Command	06/29/2016	Aloke Bordia
1.0	Added DFE suspend/resume	08/23/2016	Aloke Bordia
1.1	Added Cancel, Recover Sequences, HTTP Status Code	08/31/2016	Aloke Bordia
1.2	Added Session, DFE, Printer Inks, Jobs response and HTTP Status Code	09/19/2016	Aloke Bordia
1.3	Updated jobStarting event parameters, DFE event names changed, Job Cancel sequence	09/26/2016	Aloke Bordia
1.4	Updated printer input resource	10/14/2016	Isao Asahina
1.5	Updated Suspend/Resume/Recover/Cancel Sequences	12/13/2016	Aloke Bordia
1.6	Added DFE abort	01/11/2017	Aloke Bordia
1.7	Added Ink used parameters per job	01/26/2017	Aloke Bordia
1.8	Added printer status messages	02/28/2017	Aloke Bordia
1.9	Added printer localization	03/20/2017	Aloke Bordia
1.91	Updated imageStarting event parameters to include blank boolean	04/06/2017	Aloke Bordia
1.92	Added imageCanceling event	04/25/2017	Scott Crosby
1.93	Changed dfeRecovering to dfeRecoverPrepared	05/01/2017	Aloke Bordia
1.94	Added a reason parameter for abort	06/06/2017	Aloke Bordia
1.95	Added support for clear, gray, lightcyan, lightmagenta inks	06/29/2017	Aloke Bordia
2.0	Added Rush Print Sequences	07/18/2017	Aloke Bordia
2.01	Updated imageStarting event to include special image boolean	08/11/2017	Aloke Bordia
2.1	Added device specific information in jobStarting, new adhesive ink type	09/26/2017	Aloke Bordia
2.2	Added printer specific configuration	12/13/2017	Aloke Bordia
2.3	Removed low and empty from list of states in inputs status	01/11/2018	Isao Asahina
2.31	Clarified “imageCompleting” responses after a “jobCanceling” event. Updated TOC	01/16/2018	Scott Crosby
2.32	Added abort for a specific job and abortall	01/31/2018	Aloke Bordia
2.33	Clarification on mandatory printer ready status	02/05/2018	Aloke Bordia
2.34	Added secure websocket (wss) for events	02/22/2018	Aloke Bordia
2.35	Copy count clarification and infinite copies	02/23/2018	Aloke Bordia
2.4	Added OEM Uniformity and NozzleOut Corrections APIs and events	02/28/2018	Isao Asahina



Revision History (continued)

Version	Change Log	Date	Author
2.41	Added indication of rush print job in jobStarting event	03/05/2018	Aloke Bordia
2.42	Added inks in imageStarting event	03/13/2018	Aloke Bordia
2.43	Added device specific information in jobStarting for gloss processing and tab inserter	04/09/2018	Aloke Bordia
2.5	Added Printer Finishers	04/16/2018	Aloke Bordia
2.51	Added an optional printer initializing status	05/24/2018	Aloke Bordia
2.52	Added jobRecovering event	06/01/2018	Aloke Bordia
2.53	Clarified imageCanceling/imageCompleting response after a jobCanceling event is sent	06/01/2018	Aloke Bordia
2.54	Removed dfeRecoverPrepared event	06/05/2018	Aloke Bordia
2.55	Updated formatting	06/08/2018	Scott Crosby
2.56	Clarify the usage of last attribute available as part of imageCompleting event	06/12/2018	Aloke Bordia
2.6	Added drop size information per ink	07/02/2018	Aloke Bordia
2.7	Added optional group information to imageStarting	08/27/2018	Aloke Bordia
2.8	Added support for new inks	09/04/2018	Aloke Bordia
2.9	Added dfe info to provide version information	10/18/2018	Aloke Bordia
2.91	Modify size related fields under Set an Input Status's	11/01/2018	Isao Asahina
2.92	Added new status codes for inks and configuration and an update to the startup sequence	01/09/2019	Aloke Bordia
2.93	Added indication for generated blank in imageStarting	01/17/2019	Aloke Bordia
2.94	Added optional offset information as part of jobStarting	01/28/2019	Aloke Bordia
2.95	Added front indication in imageStarting	02/01/2019	Scott Crosby
2.96	Added optional finishing/slipsheet information in imageStarting	03/13/2019	Simeon Godinez
2.97	Removed input media details and point to product spec instead	03/25/2019	Isao Asahina
2.98	Converted "weight" in imageStarting to a "String" and added "points: {x: xxx, y: yyy}" in the "media" subsection of imageStarting.	03/26/2019	Scott Crosby
2.99	Added an optional value parameter for finishers	03/27/2019	Aloke Bordia
3.00	Added optional media information in imageStarting and removed the recently added slip-sheet information from imageStarting. Split the imageStarting table into multiple tables.	04/01/2019	Scott Crosby
3.01	Added a new status code 503 for login and inks, deprecated 412	04/10/2019	Aloke Bordia
3.02	Added an optional settings object dfe info	04/23/2019	Aloke Bordia
3.03	Added "finishing" to the imageStarting event.	05/01/2019	Scott Crosby
3.04	Add type of job to jobStarting, jobRecovering, jobCompleting and jobCanceling	05/13/2019	Aloke Bordia
3.1	Added OEM Calibration and Profiling Corrections APIs	05/21/2019	Aloke Bordia
3.2	Added optional responseRequired flag in dfe abort/abortall	06/19/2019	Aloke Bordia
3.21	Added optional state for ink status, minor clarification in Start-up sequence regarding dfe status	08/22/2019	Aloke Bordia
3.3	Added a new update and optional JSON for delete for localization	09/27/2019	Aloke Bordia



Revision History (continued)

Version	Change Log	Date	Author
3.31	Added indication for custom media and duplex finishing option in imageStarting	10/08/2019	Aloke Bordia
3.32	Added “error” element to dfeUniformityCorrectionsUpdated and dfeNozzleOutCorrectionsUpdated events to provide more detail on a failure.	10/08/2019	Scott Crosby
3.33	Added support for “id” and “responseRequired” to the DFE Status request when the “action” is “recover”.	10/21/2019	Scott Crosby
3.4	Added restart and reboot actions under dfe	11/06/2019	Aloke Bordia
3.41	Added “number” to Printer Inputs.	11/14/2019	Isao Asahina
3.5	Added Printer Outputs	11/25/2019	Aloke Bordia
3.51	Updated page number so that Revision History and Table of Contents pages are in Roman numerals and the main content starts at page 1.	01/09/2020	Scott Crosby
3.6	Added optional images to jobStarting	02/18/2020	Aloke Bordia
3.61	Added support for set and sheet “id” under “image” in imageStarting	02/18/2020	Scott Crosby
3.62	Moved sizeInPoints to media element for Printer Inputs API’s.	03/12/2020	Isao Asahina
3.63	Added optional batch indication in jobStarting	04/20/2020	Aloke Bordia
3.64	Broke up the imageStarting “finishing” table into multiple tables. Added “binding”, “subset”, and “trim” attributes to the “finishing” table. Added “binding”, “subset”, and “trim” attribute tables. Updated the “crease” attribute table.	05/18/2020	Scott Crosby
3.65	Broke up the jobStarting/jobRecovering table into multiple tables. Added “device”, “image”, and “offset” attributes tables. Added “normal”, “die-cut”, “jobSheet”, and “separatorSheet” attributes to the “image” attribute.	08/23/2020	Scott Crosby
3.66	Added “margin” attribute to the imageStarting “media” attribute table.	09/02/2020	Scott Crosby
3.7	Added colorMode in jobStarting	09/09/2020	Aloke Bordia
3.71	Updated the “perfect” elements of the “bind” attributes in imageStarting.	09/23/2020	Scott Crosby
3.8	Added scan interface	12/04/2020	Aloke Bordia
3.81	Change printhead to printBarSet and colorants to printBars in corrections, add precision	12/17/2020	Aloke Bordia
3.9	Added option to delete all inks, inputs, finishers, outputs	03/03/2021	Aloke Bordia
3.91	Added “faceup” and “sample” attributes to the imageStarting “image” attribute table.	03/11/2021	Scott Crosby
3.92	Added “subset” attribute to imageStarting “fold”, “punch”, and “staple” attribute tables.	03/16/2021	Scott Crosby
3.93	Put the “dieCut”, “jobSheet”, and “separatorSheet” attributes under a “special” attribute in the jobStarting “images” attribute table. Added “padCover” attribute to the “special” attribute. Renamed “normal” and “dieCut” to “document” and “dieLine”, accordingly.	04/11/2021	Scott Crosby
4.0	Added “display” attribute for imageCompleting	06/25/2021	Aloke Bordia
4.01	Added specialType to indicate “jobSheet”, “dieLine”, “padCover”, “separatorSheet”	08/05/2021	Aloke Bordia
4.1	Added optional secure and directQueue indication in jobStarting	09/01/2021	Nitin Arora
4.2	Added TIFF option for uniformity correction	09/09/2021	Aloke Bordia
4.21	Updated printer status description	09/27/2021	Aloke Bordia



Revision History (continued)

Version	Change Log	Date	Author
4.22	Added support for “printable” attribute to response of imageStarting event	10/07/2021	Scott Crosby
4.23	Updated the documentation for the “image” JSON object related to the “jobStarting” event	06/23/2022	Scott Crosby
4.24	Updated the “Required” column for the imageStarting “fold”, “punch”, and “staple” attributes	07/09/2022	Scott Crosby
4.25	Updated Corrections Attributes table	08/05/2022	Scott Crosby
5.0	Added Maintenance Jobs	08/26/2022	Aloke Bordia
5.1	Added jobend mode for Maintenance Jobs	10/11/2022	Aloke Bordia
5.2	Added “Details” for specific error information for failures with REST API’s	11/07/2022	Aloke Bordia
5.3	Added support for “separatorSheet” attribute to jobStarting/jobRecovering event	03/23/2023	Scott Crosby
5.4	Added support to initate a scan from the printer	04/14/2023	Aloke Bordia
5.5	Updated “separatorSheet” attribute in jobStarting/jobRecovering event	08/07/2023	Scott Crosby
5.6	Added support for “boundary” attribute to jobCanceling event	11/30/2023	Scott Crosby
5.7	Added support for “collection” attribute to “finishing” table	01/18/2024	Scott Crosby
6.0	Added support for side, resolution and ink name for corrections	02/01/2024	Aloke Bordia
6.1	Added scanSuspend and scanResume events	02/27/2024	Aloke Bordia
6.2	Added routes to delete all corrections	03/05/2024	Aloke Bordia
6.3	Added CSV_Text format for Nozzleout correction, deprecated Raw_Binary and Raw_Text	03/13/2024	Aloke Bordia
6.31	Added UCT_Text format for Uniformity correction	03/19/2024	Aloke Bordia
6.4	Added scanDone event	04/04/2024	Aloke Bordia
6.5	Added downsample option for an ink	07/19/2024	Aloke Bordia
6.51	Added jobCancelingUpdate and boundary for imageStarting and imageCompleting events	08/22/2024	Aloke Bordia
6.52	Added “duplexType” attribute in jobStarting/jobRecovering event	02/05/2025	Scott Crosby
6.53	Added dropSize attribute in imageStarting event	07/28/2025	Aloke Bordua
6.54	Added new attributes to imageCompleting event	08/11/2025	Aloke Bordia
7.0	Added support for “document” attribute to “finishing” table	10/14/2025	Scott Crosby
7.1	Added jobMode in jobStarting event	11/24/2025	Aloke Bordia
7.11	Added support for jobInterrupting and jobRestarting event. Updated “Set DFE Status Request” attributes table to include “id” and “reason” for “suspend” action.	02/09/2026	Scott Crosby

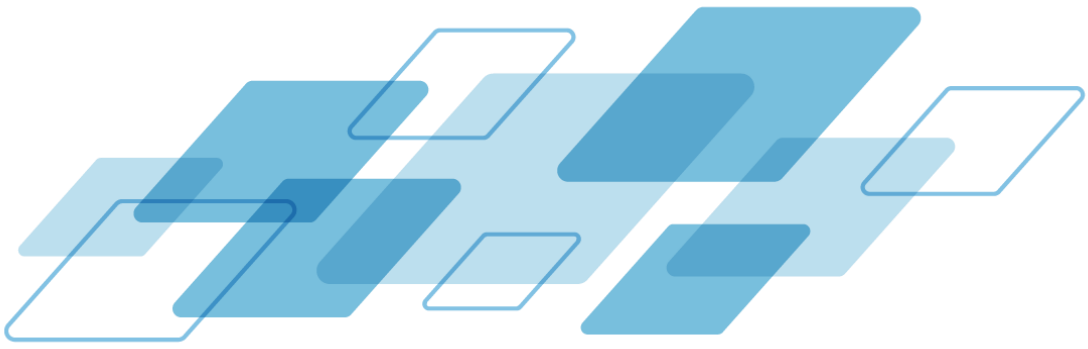


Table of Contents

1 OVERVIEW 1

1.1 CONTROL PATH 1
1.2 DATA PATH 1

2 SCENARIOS 2

2.1 SESSION 2
2.1.1 Log-in 2
2.1.2 Log-out 2
2.1.3 Configuration 2
2.1.4 Status 2
2.1.5 Start-up 2
2.1.6 Shut-down 3
2.2 PRINTING 3
2.2.1 Print a Job 3

3 EVENTS 4

3.1 DFE 4
3.2 JOB 5
3.3 IMAGE 11
3.4 SCAN 20
3.5 MAINTENANCE 22

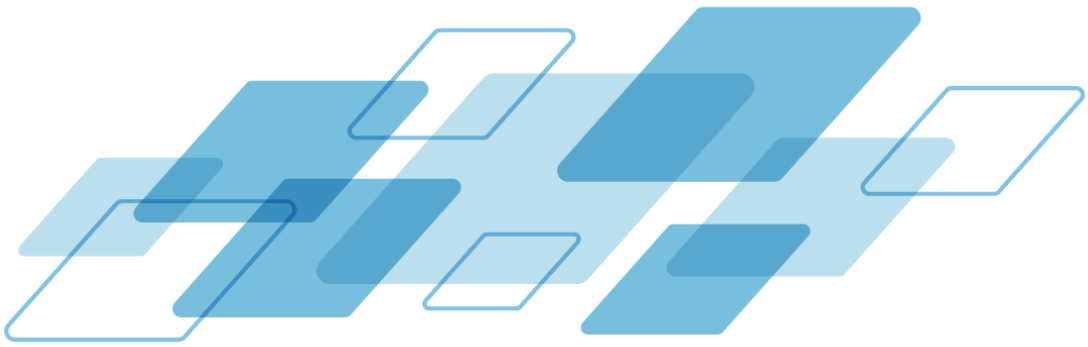
4 SEQUENCES 23

4.1 SESSION 23
4.1.1 Start-up 23
4.1.2 Shut-down 24
4.2 PRINTING 25
4.2.1 Print a Job 25
4.2.2 Suspend/Resume 26
4.2.3 Suspend/Recover/Resume 27
4.2.4 Abort 28
4.2.5 Job Cancel 29
4.2.6 Rush Print 29
4.3 SCANNING 32
4.3.1 Scan a Job 32
4.3.2 Scan Cancel 32
4.3.3 Scan Abort 33
4.4 MAINTENANCE 34
4.4.1 Setup Maintenance Jobs 34
4.4.2 Insert a Maintenance Job on Demand 34
4.4.3 Insert a Maintenance Job at the end of every Job 35

5 REST APIS 36

5.1 SESSION 36
5.2 SYSTEM 38
5.2.1 Status (DFE) 38
5.2.2 Status (Printer) 40
5.2.3 OEM Configuration (Printer) 41
5.2.4 Inks (Printer) 42
5.2.5 Inputs (Printer) 47
5.2.6 Localization (Printer) 53
5.2.7 OEM Corrections (Printer) 56

5.2.8 Finishers (Printer) *61*
5.2.9 Outputs (Printer) *62*
5.2.10 Info (DFE) *64*
5.3 JOBS 65
5.3.1 Jobs *65*
5.3.2 Scan Jobs *66*
5.3.3 Maintenance Jobs *68*
5.4 HTTP STATUS CODES 70



1 Overview

This document is intended to serve as a reference for the **Printer Controller** communication interface between Fiery DFE (Digital Front End) and a printer, from here on out known as the **FieryPath** interface. Communication between the DFE (i.e. server) and printer (i.e. client) are split into two categories:

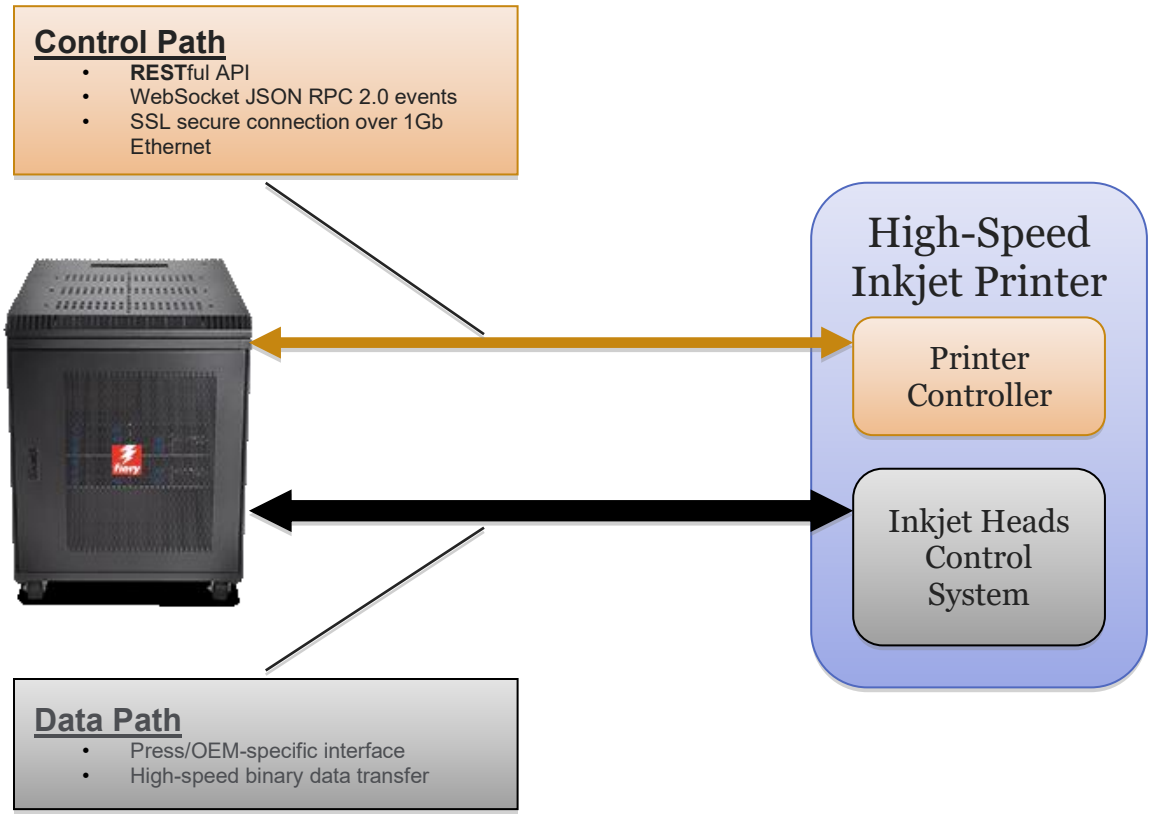
- Control Path
- Data Path

1.1 Control Path

The **Control Path** is the interface for handling all management/control functions. All client initiated communication (i.e. actions/commands) is done over SSL through a RESTful API – with the DFE's Main Blade acting as the HTTPS server and the printer acting as the client using the API defined in this document. All server initiated communication (i.e. events) is done through a WebSocket interface – with the DFE acting as the WebSocket server publishing events to connected clients. All events use the JSON RPC-2.0 protocol defined at http://www.jsonrpc.org/specification.

1.2 Data Path

The **Data Path** is the interface for handling the transfer of image data from the DFE to the printer. The **Control Path**, by receiving events and acknowledging them using the RESTful API, allows data to enter the **Data Path**. The **Data Path** is discussed in a separate document.



2 Scenarios

Scenarios are one or more **sequences**, usually resulting in a "useful" task being accomplished. The scenarios described below refer to a **Printer Controller** client. A **Printer Controller** client is responsible for handling the event-based handshaking required to transfer all the images of a job to the printer, and thus successfully print a job. In addition, a **Printer Controller** client is responsible for providing the Fiery DFE with printer configuration and status information. The scenarios are broken-up into two categories: **Session** and **Printing**.

2.1 Session

The **Session** scenarios include:

- Log-in
- Log-out
- Configuration
- Status
- Start-up
- Shut-down

2.1.1 Log-in

The **Log-in** scenario describes the steps a client must execute in order to "connect" to the system. During the **Log-in** state, the client attempts to connect to the system and obtain a "session" token using the **Log-in** sequence. This need only be executed once during the lifetime of a **FieryPath** session. After successfully completing the **Log-in** sequence, the client may execute any of the other scenarios.

2.1.2 Log-out

The **Log-out** scenario describes the steps the client must execute in order to "disconnect" from the system. During the **Log-out** state, the client proceeds to release the "session" token using the **Log-out** sequence. After successfully completing the **Log-out** sequence, the current **FieryPath** session is concluded. From this state, the only scenario that may be executed is **Log-in**.

2.1.3 Configuration

The **Configuration** scenario describes the steps a client must execute in order to communicate its "configuration" to the system. During the **Configuration** state, the client proceeds to send its capability and setting information. This scenario may be executed multiple times over the lifetime of a **FieryPath** session, based on configuration changes to the printer.

2.1.4 Status

The **Status** scenario describes the steps a client must execute in order to communicate its "status" to the system. During the **Configuration** state, the client proceeds to exchange status information. This scenario may be executed multiple times over the lifetime of a **FieryPath** session, based on status changes on the printer.

2.1.5 Start-up

The **Start-up** scenario describes the steps a client must execute in order to "connect" to the system and get to a "ready to print" state. It consists of three previously described scenarios, executed in order: **Log-in**, **Configuration**, and **Status**. During the **Log-in** state, the client attempts to connect to the system and obtain a "session" token using the **Log-in** sequence. After successfully completing the **Log-in** sequence, the client then proceeds to the **Configuration** state where capability and setting information are exchanged via the **Configuration** sequence. After successfully completing the **Configuration** sequence, the

client then proceeds to the **Status** state where status information is exchanged using the **Status** sequence. After successfully completing the **Status** sequence, the **Start-up** scenario is complete and the **FieryPath** session moves to the **Ready** state. From the **Ready** state, the client is ready to execute any of the **Printing** scenarios.

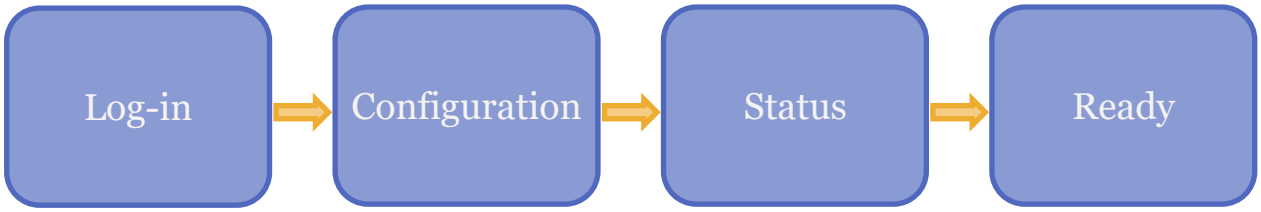


Figure 1: Start-up Scenario

2.1.6 Shut-down

The **Shut-down** scenario describes the steps the client must execute in order to "disconnect" from the system. It contains a single state: **Log-out**. During the **Log-out** state, the client proceeds to release the "session" token using the **Log-out** sequence. After successfully completing the **Log-out** sequence, the **Shut-down** scenario is complete and the current **FieryPath** session is concluded. From this state, the only scenario that may be executed is **Log-in**.

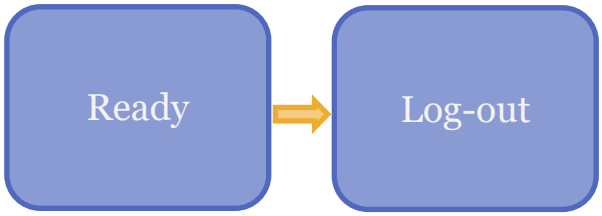


Figure 2: Shut-down Scenario

2.2 Printing

The **Printing** scenarios only include a single scenario: **Print a Job**. The **Print a Job** scenario describes the steps a client must execute in order to successfully print a job.

2.2.1 Print a Job

The **Print a Job** scenario describes the steps the client must execute in order to successfully print a job. It contains a single state: **Print a Job**. During the **Print a Job** state, the client will receive various job and image events, make sure the printer controller is ready to proceed, and then respond to those events according to the **Print a Job** sequence. After successfully completing the **Print a Job** sequence, the **FieryPath** session returns to the **Ready** state.

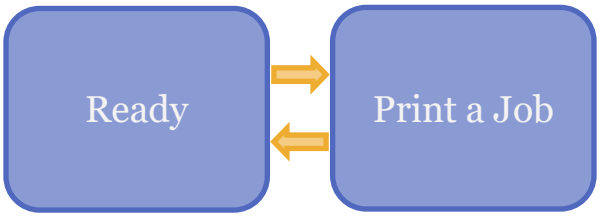


Figure 3: Print a Job Scenario

This is the only scenario which is DFE-initiated. A print request can come from one of many interfaces (**FieryPath** session, Command Workstation, Print Driver, etc).

3 Events

Below are the various events that can be received by the client via the **FieryPath** WebSocket interface. They include DFE, Job, and Image events. All events follow the JSON RPC-2.0 protocol for "notifications". Thus, they have no "id" element, but contain the required elements "jsonrpc" and "method", and may contain the "params" element if needed, as shown below:

```
{
    "jsonrpc": "2.0",
    "method": "eventName",
    "params": {
        …
    }
}
```

3.1 DFE

DFE events are used to communicate the status of the DFE to clients. Valid **DFE** events include:

Method/Event	Params	Purpose
dfeSuspended	No	Indicates the DFE has suspended any and all active jobs.
dfeResumed	No	Indicates the DFE has resumed any and all jobs.
dfeDisconnected	No	Indicates the DFE has disconnected the FieryPath interface. This usually occurs when the Fiery is requested to stop, restart, or shutdown. The printer should reinitiate the Start-up sequence in this case,
dfeAborting	No	Indicates the DFE has initiated aborting the currently printing job, a specific job or all printing jobs based on the request made by the printer.
dfeNozzleCorrectionsUpdated	Yes	Indicates the DFE has updated the nozzle out corrections. Refer to Table 2 for information regarding the attributes for this event.
dfeNozzleCorrectionsCleared	Yes	Indicates the DFE has cleared the nozzle out corrections. Refer to Table 2 for information regarding the attributes for this event.
dfeUniformityCorrectionsUpdated	Yes	Indicates the DFE has updated the uniformity corrections. Refer to Table 2 for information regarding the attributes for this event.
dfeUniformityCorrectionsCleared	Yes	Indicates the DFE has cleared the uniformity corrections. Refer to Table 2 for information regarding the attributes for this event.

Table 1: DFE Events

Element	Type	Required	Purpose
successful	Boolean	Yes	Indicates success or failure.

Table 2: Corrections Attributes

3.2 Job

Job events are used to communicate the status of a job while printing. Valid **Job** events include:

Method/Event	Params	Purpose
jobStarting	Yes	Indicates the DFE is ready to start the processing of a job. Job-specific parameters are specified in “params” element. Refer to Table 4 for information regarding the attributes for this event.
jobCompleting	Yes	Indicates the DFE has completed the processing of a job. This only means that no more imageStarting events will be generated for the given job. It does NOT indicate that ALL images in the given job have been transferred. Refer to Table 9 for information regarding the attributes for this event.
jobCanceling	Yes	Indicates that the job is being canceled. Refer to Table 10 for information regarding the attributes for this event.
jobCancelingUpdate	Yes	Indicates that the job being canceled on a boundary should instead be canceled immediately
jobRecovering	Yes	Indicates the DFE is ready to start the recovery of a job. Job-specific parameters are specified in “params” element. Refer to Table 4 for information regarding the attributes for this event. This event is sent only if jobStarting is not sent as part of recovery.
jobInterrupting	Yes	Indicates the DFE is interrupting the currently printing job in order to process a “rush” job. Job-specific parameters are specified in “params” element. Refer to Table 9 for information regarding the attributes for this event.
jobRestarting	Yes	Indicates the DFE has completed the processing of a “rush” job and is ready to restart the processing of the “interrupted” job. Job-specific parameters are specified in “params” element. Refer to Table 4 for information regarding the attributes for this event.

Table 3: Job Events

At a minimum, each **Job** event will contain a JSON object called **job** which contains an **id** (i.e. job ID). This **id** is unique per job and will not change when a job is reprinted. In addition, the **jobStarting/jobRecovering/jobRestarting** event will contain other “job-related” attributes to help the **Print Controller** decide if the job can be printed.

Valid **jobStarting/jobRecovering/jobRestarting** attributes include:

Element	Type	Required	Purpose
job	JSON Object	Yes	JSON Object containing the attributes related to the job.
id	String	Yes	Indicates the ID of the job.
type	String	Yes	Indicates the type of the job. Values include “normal”, “calibration”, “profiling”.
colorMode	String	Yes	Indicates the color mode of the job.
rush	Boolean	Yes	Indicates if this job is rush print.
copies	Number	Yes	Indicates the number of copies in the job.
duplex	Boolean	Yes	Indicates the duplex setting used in the job.
duplexType	String	No	Indicates the duplex “type” when duplex is enabled.
name	String	Yes	Indicates the name of the job.
number	Number	No	Indicates the job number relative to all jobs processed since the FieryPath interface was started-up.
reverseOrder	Boolean	Yes	Indicates if the image order setting is reverse in the job.
collate	Boolean	Yes	Indicates if the job is collated.
user	String	Yes	Indicates the name of the user.
faceup	Boolean	Yes	Indicates if the job is faceup.
device	JSON Object	Yes	JSON Object containing device specific attributes.
separatorSheet	JSON Object	Yes	JSON Object containing separator sheet specific attributes.
images	JSON Object	No	JSON Object containing “image” information known at the time of this event. Since the DFE may still be generating images for this job, the information may be incomplete.
offset	JSON Object	No	JSON Object containing offset specific attributes.
batch	Boolean	No	Indicates if the job is part of a batch.
directQueue	Boolean	No	Indicates if the job is directQueue
secure	Boolean	No	Indicates if the job is secure
jobMode	Number	Yes	Indicates if jobMode is enabled and data is sent only for the 1st copy

Table 4: jobStarting/jobRecovering/jobRestarting Attributes

Element	Type	Required	Purpose
screenType	String	Yes	Indicates the device screen type.
colorMode	String	No	Indicates the device color mode.
additionalColors	JSON Array	No	JSON Array containing the additional colors supported by the device.
glossProcessing	String	No	Indicates the gloss processing mode.
tabInserter	String	No	Indicates the tab inserter type.

Table 5: jobStarting/jobRecovering/jobRestarting “device” Attributes

Element	Type	Required	Purpose
enabled	Boolean	Yes	Indicates if “separator sheet” is enabled.
copy	JSON Object	No	JSON Object containing “separator sheet” information per “copy”.
before	Boolean	Yes	Indicates if inserting “separator sheet” before the “copy” is enabled.
after	Boolean	Yes	Indicates if inserting “separator sheet” after the “copy” is enabled.
frequency	Number	Yes	Indicates the “separator sheet” frequency.
set	JSON Object	No	JSON Object containing “separator sheet” information per “set”.
before	Boolean	Yes	Indicates if inserting “separator sheet” before the “set” is enabled.
after	Boolean	Yes	Indicates if inserting “separator sheet” after the “set” is enabled.
frequency	Number	Yes	Indicates the “separator sheet” frequency.
sheet	JSON Object	No	JSON Object containing “separator sheet” information per “sheet”.
before	Boolean	Yes	Indicates if inserting “separator sheet” before the “sheet” is enabled.
after	Boolean	Yes	Indicates if inserting “separator sheet” after the “sheet” is enabled.
frequency	Number	Yes	Indicates the “separator sheet” frequency.
job	JSON Object	No	JSON Object containing “separator sheet” information per “job”.
before	Boolean	Yes	Indicates if inserting “separator sheet” before the “job” is enabled.
after	Boolean	Yes	Indicates if inserting “separator sheet” after the “job” is enabled.
frequency	Number	Yes	Indicates the “separator sheet” frequency.

Table 6: jobStarting/jobRecovering/jobRestarting “separatorSheet” Attributes

Element	Type	Required	Purpose
document	JSON Object	No	JSON Object containing “document” (i.e. user’s job) image information.
total	Number	No	Indicates the total number of “document” images in the job.
special	JSON Object	No	JSON Object containing “special” image information.
total	Number	No	Indicates the total number of “special” images in the job.
dieLine	JSON Object	No	JSON Object containing “die line” image information.
total	Number	No	Indicates the total number of “die line” images (i.e. sum of “before” and “after”) in the job.
before	Number	No	Indicates the number of “die line” images printing before the “document” images in the job.
after	Number	No	Indicates the number of “die line” images printing after the “document” images in the job.
jobSheet	JSON Object	No	JSON Object containing “job sheet” (i.e. banner page) image information.
total	Number	No	Indicates the total number of “job sheet” images (i.e. sum of “before” and “after”) in the job.
before	Number	No	Indicates the number of “job sheet” images printing before the “document” images in the job.
after	Number	No	Indicates the number of “job sheet” images printing after the “document” images in the job.
padCover	JSON Object	No	JSON Object containing “pad cover” image information.
total	Number	No	Indicates the total number of “pad cover” images in the job.
separatorSheet	JSON Object	No	JSON Object containing “separator sheet” (i.e. insert) image information.
total	Number	No	Indicates the total number of “separator sheet” images (i.e. sum of “before” and “after”) in the job.
before	Number	No	Indicates the number of “separator sheet” images printing before the “document” images in the job.
after	Number	No	Indicates the number of “separator sheet” images printing after the “document” images in the job.

Table 7: jobStarting “images” Attributes

Element	Type	Required	Purpose
boundary	String	Yes	Indicates the offset boundary.
frequency	Number	Yes	Indicates the offset frequency.

Table 8: jobStarting/jobRecovering/jobRestarting "offset" Attributes

Below is a sample **jobStarting/jobRecovering/jobRestarting** event:

```
{
    "jsonrpc": "2.0",
    "method": "jobStarting" or "jobRecovering" or "jobRestarting",
    "params": {
        "job": {
            "id": "00000000.573F5290.0",
            "type": normal",
            "colorMode": "CMYK",
            "rush": false,
            "copies": 2,
            "duplex": false,
            "name": "test_page.pdf",
            "reverseOrder": false,
            "collate": true,
            "user": "Username",
            "faceup": true,
            "device": {
                "screenType": "Stochastic",
                "colorMode": "",
                "additionalColors": [],
                "glossProcessing": "Auto",
                "tabInserter": "OnFirstSheetJob"
            },
            "images": {
                "document": {"total": 20},
                "special": {"total": 1, "jobSheet": {"total": 1, "before": 1}},
            },
            "offset": {"boundary": "copies", "frequency": 1}
        }
    }
}
```

The following should be noted about "**copies**":

1. This reflects the number of copies in the job (it does not include any special images like banner, die line, inserts, etc.)
2. A value of 0 indicates **infinite** copies

The following should be noted about "**images**":

1. This optional value is only available in the **jobStarting** event and only when the various elements can be accurately determined.
2. This object reflects the number of images in the job, but does not currently include any periodic "separator" sheets (only "separator" sheets that would be inserted "before" and/or "after" a job).

Valid **jobCompleting/jobInterrupting** attributes include:

Element	Type	Required	Purpose
job	JSON Object	Yes	JSON Object containing the attributes related to the job.
id	String	Yes	Indicates the ID of the job.
type	String	Yes	Indicates the type of the job. Values include “normal”, “calibration”, “profiling”
name	String	Yes	Indicates the name of the job.
user	String	Yes	Indicates the name of the user.

Table 9: jobCompleting/jobInterrupting Attributes

Below is a sample **jobCompleting/jobInterrupting** event:

```
{
    “jsonrpc”: “2.0”,
    “method”: “jobCompleting” or “jobInterrupting”,
    “params”: {
        “job”: {
            “id”: “00000000.573F5290.0",
            “type”: normal”,
            “name”: “test_page.pdf”,
            “user”: “Username”
        }
    }
}
```

Valid **jobCanceling/jobCancelingUpdate** attributes include:

Element	Type	Required	Purpose
job	JSON Object	Yes	JSON Object containing the attributes related to the job.
id	String	Yes	Indicates the ID of the job.
type	String	Yes	Indicates the type of the job. Values include “normal”, “calibration”, “profiling”
name	String	Yes	Indicates the name of the job.
user	String	Yes	Indicates the name of the user.
responseRequired	Boolean	Yes	Indicates if the printer needs to respond to this event
boundary	Boolean	No	Indicates if cancel on a “boundary” is enabled. If enabled, the client should allow ALL outstanding images to be successfully printed before responding to this event. For jobCancelingUpdate, this attribute will not be set

Table 10: jobCanceling/jobCancelingUpdate Attributes

Below is a sample **jobCanceling/jobCancelingUpdate** event:

```
{
    “jsonrpc”: “2.0”,
    “method”: “jobCanceling”,
    “params”: {
        “job”: {
            “id”: “00000000.573F5290.0",
            “type”: normal”,
            “name”: “test_page.pdf”,
            “user”: “Username”,
            “responseRequired”: true
        }
    }
}
```

3.3 Image

Image events are used to communicate the status of an image within a job while printing. Valid **Image** events include:

Method/Event	Params	Purpose
imageStarting	Yes	Indicates the DFE is ready to process and then transfer an image. Refer to Table 12 for information regarding the attributes for this event.
imageCompleting	Yes	Indicates the DFE has completed the transfer of an image. Refer to Table 29 for information regarding the attributes for this event.
imageCanceling	Yes	Indicates that the image is being canceled because the job is canceling. Refer to Table 30 for information regarding the attributes for this event.

Table 11: Image Events

The **imageStarting** event is an indication from the DFE to the printer that it is ready to start processing an image. The printer should respond with a "**start**" to this event as quickly as possible – any significant delay could prevent Fiery DFE from processing and transferring image data at a rate necessary to maintain the printer's required speed. Once the printer responds, the DFE will do the necessary processing and then transfer the image. The printer should make no particular assumption about the time it will take for the DFE to transfer the image based on when it responded with a "**start**" to the **imageStarting** event.

Each **imageStarting** event contains a JSON object called **job** which contains an **id** (i.e. job ID) and another JSON object called **image** which contains an **id** (i.e. image ID). In addition, the **imageStarting** event will contain other "image-related" attributes to help the **printer** decide if the image can be printed. If this image is part of a larger group, it will contain an additional JSON object with the required group information. Valid **imageStarting** attributes include the following as shown below:

Element	Type	Required	Purpose
job	JSON Object	Yes	JSON Object containing the attributes related to the job.
id	String	Yes	Indicates the ID of the job.
image	JSON Object	Yes	JSON Object containing the attributes related to the image (printable data/content).
media	JSON Object	Yes	JSON Object containing the attributes related to the media (physical page/substrate).
group	JSON Object	No	JSON Object containing the group information. It will be available when this image is part of a group that comprises a larger image.
finishing	JSON Object	No	JSON Object containing the finishing information. It will be available when this image has one or more finishing "options" enabled.

Table 12: imageStarting Attributes

Element	Type	Required	Purpose
id	String	Yes	Indicates the ID of the image.
bitsPerColor	Number	Yes	Indicates the number of bits/colorant in the image.
colorMode	String	Yes	Indicates the color mode of the image.
inks	String	Yes	Indicates the inks present in the image.
copy	Number	Yes	Indicates the copy number of the image.
mirror	Boolean	Yes	Indicates the mirror setting used in the image.
number	Number	No	Indicates the image number relative to all images processed since the FieryPath interface was started-up.
page	Number	Yes	Indicates the page number of the image.
blank	Boolean	Yes	Indicates if the image is blank.
pixels	JSON Object	Yes	JSON Object containing the attributes related to the dimensions of the image in pixels.
x	Number	Yes	Indicates the number of pixels in the image along the X-axis.
y	Number	Yes	Indicates the number of pixels in the image along the Y-axis.
resolution	JSON Object	Yes	JSON Object containing the attributes related to the resolution of the image dots per inch.
x	Number	Yes	Indicates the resolution of the image along the X-axis.
y	Number	Yes	Indicates the resolution of the image along the Y-axis.
first	Boolean	Yes	Indicates if this is the first image in the job.
last	Boolean	Yes	Indicates if this is the last image in the job.
boundary	Boolean	No	Indicates if this is the last image for a cancel on boundary job
sample	Boolean	Yes	Indicates if the image is a sample.
special	Boolean	Yes	Indicates if this image is special and not part of the regular job.
specialType	String	No	Indicates the type of image when special is true. Valid values are “jobSheet”, “dieLine”, “padCover”, “separatorSheet”.
dropSize	Number	No	Indicates the number of drop sizes
faceup	Boolean	Yes	Indicates if the image is faceup.
front	Boolean	Yes	Indicates if this image is the front side.
document	Boolean	Yes	Indicates if the image is part of the “document” (i.e. user’s job) and not a generated one e.g. if a blank is generated as the last image for a duplex job with odd number of pages
set	JSON Object	Yes	JSON Object containing the attributes related to the set.
id	String	Yes	Indicates the ID of the set.
number	Number	No	Indicates the set number relative to all sets processed since the FieryPath interface was started-up.
sheet	JSON Object	Yes	JSON Object containing the attributes related to the sheet.
id	String	Yes	Indicates the ID of the sheet.
number	Number	No	Indicates the sheet number relative to all sheets processed since the FieryPath interface was started-up.

Table 13: imageStarting “image” Attributes

Element	Type	Required	Purpose
id	String	Yes	Indicates the ID of the media.
coating	JSON Object	No	JSON Object containing the attributes related to the media's coating.
back	Boolean	No	Indicates if the back of the media is coated.
front	Boolean	No	Indicates if the front of the media is coated.
type	String	No	Indicates the media coating.
color	String	No	Indicates the media color.
custom	Boolean	Yes	Indicates if custom media is used.
finish	String	No	Indicates the media finish.
input	String	No	Indicates the input setting used for the media.
name	String	No	Indicates the name of the media.
output	String	No	Indicates the output setting used for the media.
pixels	JSON Object	Yes	JSON Object containing the attributes related to the dimensions of the media in pixels.
margin	JSON Object	Yes	JSON Object containing the attributes related to the dimensions of the media margins in pixels.
bottom	Number	Yes	Indicates the bottom margin in pixels.
left	Number	Yes	Indicates the left margin in pixels.
right	Number	Yes	Indicates the right margin in pixels.
top	Number	Yes	Indicates the top margin in pixels.
x	Number	Yes	Indicates the number of pixels in the media along the X-axis.
y	Number	Yes	Indicates the number of pixels in the media along the Y-axis.
points	JSON Object	Yes	JSON Object containing the attributes related to the dimensions of the media in points.
x	Number	Yes	Indicates the number of points in the media along the X-axis.
y	Number	Yes	Indicates the number of points in the media along the Y-axis.
printed	Boolean	No	Indicates if the media is pre-printed.
punched	String	No	Indicates the media pre-punch hole type.
type	String	Yes	Indicates the media type.
weight	String	No	Indicates the media weight.

Table 14: imageStarting "media" Attributes

Element	Type	Required	Purpose
id	Number	Yes	Indicates the ID of the group.
pixels	JSON Object	Yes	JSON Object containing the attributes related to the dimensions of the group in pixels.
x	Number	Yes	Indicates the number of pixels in the group along the X-axis.
y	Number	Yes	Indicates the number of pixels in the group along the Y-axis.
images	JSON Object	Yes	JSON Object containing the attributes related to the images in the group.
x	Number	Yes	Indicates the number of images in the group along the X-axis. Currently, this value will be set to 1.
y	Number	Yes	Indicates the number of images in the group along the Y-axis.
location	JSON Object	Yes	JSON Object containing the attributes related to the location of the image in the group.
x	Number	Yes	Indicates the location of the image in the group along the X-axis. Currently, this value will be set to 1.
y	Number	Yes	Indicates the location of the image in the group along the Y-axis.

Table 15: imageStarting "group" Attributes

Element	Type	Required	Purpose
binding	JSON Object	No	JSON Object containing the attributes related to the “binding” option.
collection	JSON Object	No	JSON Object containing the attributes related to the “collection” option. For collated job, "collection" is all pages of a copy (e.g. copy 3 of pages 1-n). For uncollated job, "collection" is all copies of a page (e.g. copies 1-n of page 3).
crease	JSON Object	No	JSON Object containing the attributes related to the “crease” option.
document	JSON Object	No	JSON Object containing the attributes related to the “document” option.
duplex	JSON Object	No	JSON Object containing the attributes related to the “duplex” option.
fold	JSON Object	No	JSON Object containing the attributes related to the “fold” option.
pad	JSON Object	No	JSON Object containing the attributes related to the “pad printing” option.
punch	JSON Object	No	JSON Object containing the attributes related to the “punch” option.
staple	JSON Object	No	JSON Object containing the attributes related to the “staple” option.
subset	JSON Object	No	JSON Object containing the attributes related to the “subset” option.
tab	JSON Object	No	JSON Object containing the attributes related to the “tab” option.
trim	JSON Object	No	JSON Object containing the attributes related to the “trim” option.

Table 16: imageStarting “finishing” Attributes

Element	Type	Required	Purpose
comb	JSON Object	No	JSON Object containing the attributes related to the “comb-binding” option.
type	String	Yes	Indicates the comb “type”.
edge	String	No	Indicates the comb “edge”.
number	Number	No	Indicates the number of holes in the “comb”.
perfect	JSON Object	No	JSON Object containing the attributes related to the “perfect-binding” option.
type	String	Yes	Indicates the perfecting “type”.
body	JSON Object	No	JSON Object containing the attributes related to the “body” option.
center	String	No	Indicates the body “size”.
image	JSON Object	No	JSON Object containing the attributes related to the “image” option.
shift	JSON Object	Yes	JSON Object containing the attributes related to the “shift” option.
horizontal	Number	Yes	Indicates the “horizontal” image “shift”.
vertical	Number	Yes	Indicates the “vertical” image “shift”.
margin	Number	No	Indicates the body “margin”.
scale	String	No	Indicates the body “scale”.
cover	JSON Object	No	JSON Object containing the attributes related to the “cover” option.
back	String	Yes	Indicates the cover “back”.
front	String	Yes	Indicates the cover “front”.
crease	String	No	Indicates the cover “crease”.
image	JSON Object	No	JSON Object containing the attributes related to the “image” option.
shift	JSON Object	Yes	JSON Object containing the attributes related to the “shift” option.
horizontal	Number	Yes	Indicates the “horizontal” image “shift”.
vertical	Number	Yes	Indicates the “vertical” image “shift”.
media	JSON Object	No	JSON Object containing the attributes related to the “media” option.
input	String	No	Indicates the input setting used for the “cover” media.
size	String	No	Indicates the cover “size”.
spine	JSON Object	No	JSON Object containing the attributes related to the “spine” option.
content	String	No	Indicates the spine “content”.
width	Number	No	Indicates the spine “width”.
finish	JSON Object	No	JSON Object containing the attributes related to the “finish” option.
size	JSON Object	Yes	JSON Object containing the attributes related to the “size” option.
length	Number	Yes	Indicates the finish “length”.
width	Number	Yes	Indicates the finish “width”.
scale	String	No	Indicates the perfecting “scale”.

Table 17: imageStarting “binding” Attributes

Element	Type	Required	Purpose
begin	Boolean	Yes	Indicates the “start” of a collection.
end	Boolean	Yes	Indicates the “end” of a collection.

Table 18: imageStarting “collection” Attributes

Element	Type	Required	Purpose
type	String	Yes	Indicates the crease “type”.
creases	JSON Array	No	JSON Array containing the custom “crease” option(s).
direction	String	No	Indicates the crease “direction”.
position	Number	No	Indicates the crease “position”.
pressure	Number	No	Indicates the crease “pressure”.

Table 19: imageStarting “crease” Attributes

Element	Type	Required	Purpose
begin	Boolean	Yes	Indicates the “first” page in the “document” (i.e. user’s job).
end	Boolean	Yes	Indicates the “last” page in the “document” (i.e. user’s job).

Table 20: imageStarting “document” Attributes

Element	Type	Required	Purpose
type	String	Yes	Indicates the duplex “type”.

Table 21: imageStarting “duplex” Attributes

Element	Type	Required	Purpose
type	String	No	Indicates the fold “type”.
direction	String	No	Indicates the fold “direction”.
saddle	JSON Object	No	JSON Object containing the attributes related to the “saddle-fold” option.
adjustment	Number	No	Indicates the adjustment “level”.
number	Number	No	Indicates the number of sheets in the “saddle-fold”.
tri	JSON Object	No	JSON Object containing the attributes related to the “tri-fold” option.
number	Number	No	Indicates the number of sheets in the “tri-fold”.
subset	JSON Object	No	JSON Object containing the attributes related to a “fold” subset.
begin	Boolean	No	Indicates the “start” of a “fold” subset.
end	Boolean	No	Indicates the “end” of a “fold” subset.

Table 22: imageStarting “fold” Attributes

Element	Type	Required	Purpose
enabled	Boolean	Yes	Indicates if “pad printing” is enabled.

Table 23: imageStarting “pad” Attributes

Element	Type	Required	Purpose
edge	String	No	Indicates the punch “edge”.
type	String	No	Indicates the punch “type”.
subset	JSON Object	No	JSON Object containing the attributes related to a “punch” subset.
begin	Boolean	No	Indicates the “start” of a “punch” subset.
end	Boolean	No	Indicates the “end” of a “punch” subset.

Table 24: imageStarting “punch” Attributes

Element	Type	Required	Purpose
type	String	No	Indicates the staple “type”.
pitch	String	No	Indicates the staple “pitch”.
subset	JSON Object	No	JSON Object containing the attributes related to a “staple” subset.
begin	Boolean	No	Indicates the “start” of a “staple” subset.
end	Boolean	No	Indicates the “end” of a “staple” subset.

Table 25: imageStarting “staple” Attributes

Element	Type	Required	Purpose
begin	Boolean	Yes	Indicates the “start” of a subset.
end	Boolean	Yes	Indicates the “end” of a subset.

Table 26: imageStarting “subset” Attributes

Element	Type	Required	Purpose
type	String	Yes	Indicates the tab “type”. Valid values include: “insert” or “eject”.
number	Number	No	Indicates the tab “sequence/cycle” number.

Table 27: imageStarting “tab” Attributes

Element	Type	Required	Purpose
type	String	Yes	Indicates the trim “type”.
position	JSON Object	No	JSON Object containing the attributes related to the “position” option.
horizontal	Number	No	Indicates the “horizontal” position.
vertical	Number	No	Indicates the “vertical” position.
finish	JSON Object	No	JSON Object containing the attributes related to the “finish” option.
size	String	Yes	Indicates the finish “size”.
length	Number	No	Indicates the “custom” finish “length”.
width	Number	No	Indicates the “custom” finish “width”.
top	Number	No	Indicates the “top” trim amount.
bottom	Number	No	Indicates the “bottom” trim amount.
crossProcess	Boolean	No	Indicates if “cross process” direction trim is enabled.
spine	String	No	Indicates the “spine” burnishing.

Table 28: imageStarting “trim” Attributes

The following shows a sample of **imageStarting** event:


```
{
    “jsonrpc”: “2.0”,
    “method”: “imageStarting”,
    “params”: {
        “job”: {
            “id”: “00000000.573F5290.0",
            “image”: {
                “id”: “1”,
                “bitsPerColor”: 8,
                “colorMode”: “CMYK”,
                 “inks”: “Cyan;Magenta;Yellow;Black”,
                “copy”: 1,
                “mirror”: false,
                “page”: 1,
                “blank”: false,
                “pixels”: {“x”: 13011, “y”: 10011},
                “resolution”: {“x”: 1200, “y”: 1200},
                “first”: true,
                “last”: false,
                “sample”: false,
                “special”: false,
                “faceup”: false,
                “front”: true,
                “document”: true
            },
            “media”: {
                “id”: “5”,
                “name”: “Letter”,
                “pixels”: {“x”: 13200, “y”: 10011},
                “points”: {“x”: 612.00, “y”: 792.00},
                “custom”: false,
                “type”: “HighQuality”,
                “weight”: “64_105”
            },
            “group”: {
                “id”: “5”,
                “pixels”: {“x”: 13011, “y”: 1001100},
                “images”: {“x”: 1, “y”: 100},
                “location”: {“x”: 1, “y”: 25}
            },
            “finishing”: {
                 “tab”: {“type”: “insert”, “number”: 1}
            }
        }
    }
}
```


The **imageCompleting** event is an indication from the DFE to the printer that the transfer of an image to the printer has been completed. It is only at this stage that the printer can safely assume the image is available for printing. As noted earlier, this assumption does not apply to the response of the **imageStarting** event.

In order to determine if all images in the job have been successfully transferred, the printer can use the **imageCompleting** events received. If the printer has chosen to send responses to **imageStarting** events in sequence, it can use the “**last**” attribute available in the **imageCompleting** event as an indication that all images have been transferred for the job and the printer can send a response to **jobCompleting** event once the images have been printed. However, if the printer has not sent

the responses to **imageStarting** events in sequence, it cannot use the "**last**" attribute as an indication that all images have been transferred for the job. It should instead keep a count of responses sent for **imageStarting** events and match that with the number of **imageCompleting** events received to know when all images for the job have been successfully transferred.

Valid **imageCompleting** attributes include:

Element	Type	Required	Purpose
job	JSON Object	Yes	JSON Object containing the attributes related to the job.
id	String	Yes	Indicates the ID of the job.
image	JSON Object	Yes	JSON Object containing the attributes related to the image.
id	String	Yes	Indicates the ID of the image.
copy	Number	Yes	Indicates the copy number of the image.
blank	Boolean	Yes	Indicates if the image is blank.
first	Boolean	Yes	Indicates if this is the first image in the job
last	Boolean	Yes	Indicates if this is the last image in the job
faceup	Boolean	Yes	Indicates if the image is faceup.
front	Boolean	Yes	Indicates if this image is the front side.
boundary	Boolean	No	Indicates if this is the last image for a cancel on boundary job

Table 29: imageCompleting Attributes

The following shows a sample of **imageCompleting** event:

```
{   "jsonrpc": "2.0",
    "method": "imageCompleting",
    "params": {
        "job": {
            "id": "00000000.573F5290.0",
            "image": {"id": "1", "first": true, "last": false}
        }
    }
}
```

If the printer has initiated a "**start**" for an image based on an **imageStarting** event and if the job is canceled during this window, the printer will receive either an **imageCanceling** or **imageCompleting** event for this image. Valid **imageCanceling** attributes include:

Element	Type	Required	Purpose
job	JSON Object	Yes	JSON Object containing the attributes related to the job.
id	String	Yes	Indicates the ID of the job.
image	JSON Object	Yes	JSON Object containing the attributes related to the image.
id	String	Yes	Indicates the ID of the image.

Table 30: imageCanceling Attributes

The following shows a sample of **imageCanceling** event:

```
{   "jsonrpc": "2.0",
    "method": "imageCanceling",
    "params": {
        "job": {
            "id": "00000000.573F5290.0",
            "image": {"id": "1", "first": true, "last": false}
        }
    }
}
```

3.4 Scan

Scan events are used to communicate the status of a job while scanning. Valid **Scan** events include:

Method/Event	Params	Purpose
scanJobStarting	Yes	Indicates the DFE is ready to start the processing of a scan job. Job-specific parameters are specified in “params” element. Refer to Table 32 for information regarding the attributes for this event.
scanImageCreated	Yes	Indicates the DFE has completed the creation of a scanned image. Refer to Table 33 for information regarding the attributes for this event.
scanJobCompleted	Yes	Indicates the DFE has completed the processing of a scan job. Refer to Table 34 for information regarding the attributes for this event.
scanJobCanceling	Yes	Indicates that the DFE has initiated a cancel for the scan job in progress. Refer to Table 35 for information regarding the attributes for this event.
scanSuspend	No	Indicates that a suspend has been initiated for scan
scanResume	No	Indicates that a resume has been initiated for scan
scanDone	No	Indicates a scan finish event

Table 31: Scan Job Events

At a minimum, each **Scan** event will contain a JSON object called **job** which contains an **id** (i.e. job ID). This **id** is unique per job. In addition, the **scanJobStarting** event will contain other “scan-related” attributes.

Valid **scanJobStarting** attributes include:

Element	Type	Required	Purpose
job	JSON Object	Yes	JSON Object containing the attributes related to the scan job.
id	String	Yes	Indicates the ID of the scan job.
settings	JSON Object	Yes	JSON Object containing scan specific attributes. Refer to product specific documentation for details.

Table 32: scanJobStarting Attributes

Below is a sample **scanJobStarting** event:

```
{
    “jsonrpc”: “2.0”,
    “method”: “scanJobStarting”,
    “params”: {
        “job”: {
            “id”: “Fiery:10",
             “settings”: {
            }
        }
    }
}
```

Valid **scanImageCreated** attributes include:

Element	Type	Required	Purpose
job	JSON Object	Yes	JSON Object containing the attributes related to the scan image created.
id	String	Yes	Indicates the ID of the scan job.
filepath	String	Yes	Indicates the fully qualified path to the image.

Table 33: scanImageCreated Attributes

Below is a sample **scanImageCreated** event:

```
{
    “jsonrpc”: “2.0”,
    “method”: “scanImageCreated”,
    “params”: {
        “job”: {
            “id”: “Fiery:10",
            “filepath”: “C:\Temp\Test.jpg”
        }
    }
}
```

Valid **scanJobCompleted** attributes include:

Element	Type	Required	Purpose
job	JSON Object	Yes	JSON Object containing the attributes related to the scan job.
id	String	Yes	Indicates the ID of the scan job.

Table 34: scanJobCompleted Attributes

Below is a sample **scanJobCompleted** event:

```
{
    “jsonrpc”: “2.0”,
    “method”: “scanJobCompleted”,
    “params”: {
        “job”: {
            “id”: “Fiery:10"
        }
    }
}
```

Valid **scanJobCanceling** attributes include:

Element	Type	Required	Purpose
job	JSON Object	Yes	JSON Object containing the attributes related to the scan job.
id	String	Yes	Indicates the ID of the scan job.

Table 35: scanJobCanceling Attributes

Below is a sample **scanJobCanceling** event:

```
{
    “jsonrpc”: “2.0”,
    “method”: “scanJobCanceling”,
    “params”: {
        “job”: {
            “id”: “Fiery:10"
        }
    }
}
```

Note that **scanSuspend**, **scanResume** and **scanDone** do not have any additional attributes.

3.5 Maintenance

Maintenance events are used to communicate the status of a maintenance job while printing. Valid **Maintenance** events include:

Method/Event	Params	Purpose
maintenanceImageCompleting	Yes	Indicates the DFE has completed the transfer of a maintenance image. Refer to Table 37 for information regarding the attributes for this event.

Table 36: Maintenance Job Events

The **maintenanceImageCompleting** event is an indication from the DFE to the printer that the transfer of a maintenance image to the printer has been completed. It is only at this stage that the printer can safely assume the image is available for printing.

These events will be sent for each page in the maintenance job when the printer sends a request to the DFE to insert a maintenance job or when the maintenance job is setup to be inserted at the end of every job that is printed.

Valid **maintenanceImageCompleting** attributes include:

Element	Type	Required	Purpose
job	JSON Object	Yes	JSON Object containing the attributes related to the maintenance job.
name	String	Yes	Indicates the name of the maintenance job.
image	JSON Object	Yes	JSON Object containing the attributes related to the image.
id	String	Yes	Indicates the ID of the image.
after	String	Yes	Indicates the image after which this maintenance image was inserted.
last	Boolean	Yes	Indicates if this is the last image in the maintenance job

Table 37: maintenanceImageCompleting Attributes

The following shows a sample of **imageCompleting** event:

```
{
    “jsonrpc”: “2.0”,
    “method”: “maintenanceImageCompleting”,
    “params”: {
        “job”: {
            “name”: “MainteanceJob1.pdf",
            “image”: {“id”: “1”, “after”: “300”, “last”: true}
        }
    }
}
```

4 Sequences

Sequences are a set of one or more FieryPath API **actions**, and possibly one or more **events**, resulting in a "useful" task being accomplished. Similar to **scenarios**, the sequences described below are broken-up into three categories: **Session, Printing and Scanning.**

A few points to note pertaining to the sequence diagrams described in the subsequent sections:

- All APIs require a "session" token to be provided in the body. This token is received during the **Log-in** state of the **Start-up** sequence described later.
- All APIs will return HTTP Status Codes (e.g. 200 indicating success or other standard codes). Only those APIs which also return data in the body of the response are shown in the sequence diagrams.
- Dotted lines are used to indicate optional actions.
- Loops (1…n) are used to indicate an API which can be used more than once pertaining to that specific operation.
- JSON format is used for all API's that require input to be provided. Specific details are provided with each action in the **REST APIs** section.

4.1 Session

The **Session** sequences include:

- Start-up
- Shut-down

4.1.1 Start-up

The **Start-up** sequence describes the steps a client must execute in order to "connect" to the system. The first action in this sequence is the **Log-in** action. When a client performs this action with the appropriate credentials, a valid "session" token is returned. This token is required to access all other FieryPath **resources** – including opening a **mandatory** WebSocket connection in order to receive FieryPath **events**. Therefore, the **Start-up** sequence should be the first action performed by a client. This need only be executed once during the lifetime of a **FieryPath** session. The only exception is when the printer receives a "**dfeDisconnected**" event. In that case, it needs to reinitiate the **Start-up** sequence. The very first thing the printer needs to do after a successful **Log-in** action and establishing the WebSocket connection is to get the **dfe status**. The printer should not perform any other action until the dfe status is either "**connected**" or "**ready**". Once the dfe reports its status as either "connected" or "ready", the printer can move onto the Configuration sequence. It can also optionally set its state to "**initializing**" during the **Start-up** sequence. This will allow a graceful recovery in the case where the client restarts for any reason abruptly.

As part of this sequence, the printer should set its configuration including status as shown. Once it is done with setting configuration, it is **mandatory** that the printer set its status to "**ready**". If it does not do that, it will not receive any printing events.

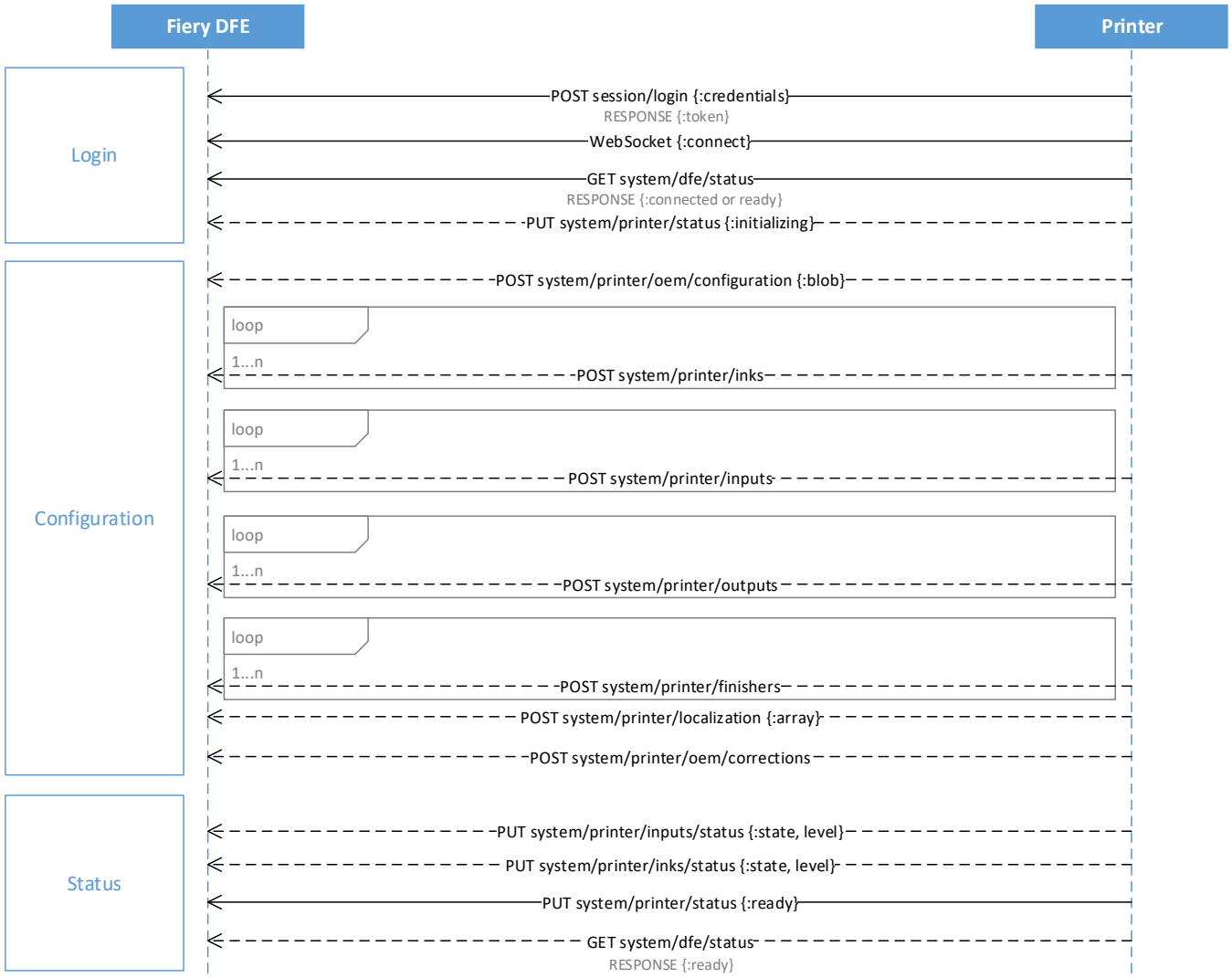


Figure 4: Start-up Sequence

4.1.2 Shut-down

The **Shut-down** sequence describes the steps the client must execute in order to “disconnect” from the system at the end of a session. The only mandatory action in this sequence is the **Log-out** action. This notifies the server that the client is disconnecting and causes the server to disassociate the “session” token, close the associated WebSocket connection and free-up any other resources allocated for the session. This need only be executed once during the lifetime of a **FieryPath** session. It is necessary that each **Start-up** sequence have a corresponding **Shut-down** sequence in order to not waste resources on Fiery DFE.



Figure 5: Shut-down Sequence

4.2 Printing

The Printing sequences include the following sequences: Print a Job, Suspend/Resume, Suspend/Recover, Abort and Job Cancel.

4.2.1 Print a Job

The **Print a Job** sequence describes the steps a client must execute in order to successfully print a job. The events initiated from Fiery DFE require a corresponding action to be taken by the printer as shown in the sequence below. The Fiery DFE and printer sequences will happen in parallel. Each printer action is a response to a specific Fiery DFE event (color coding is used in the diagram to visually associate an action with the corresponding event).

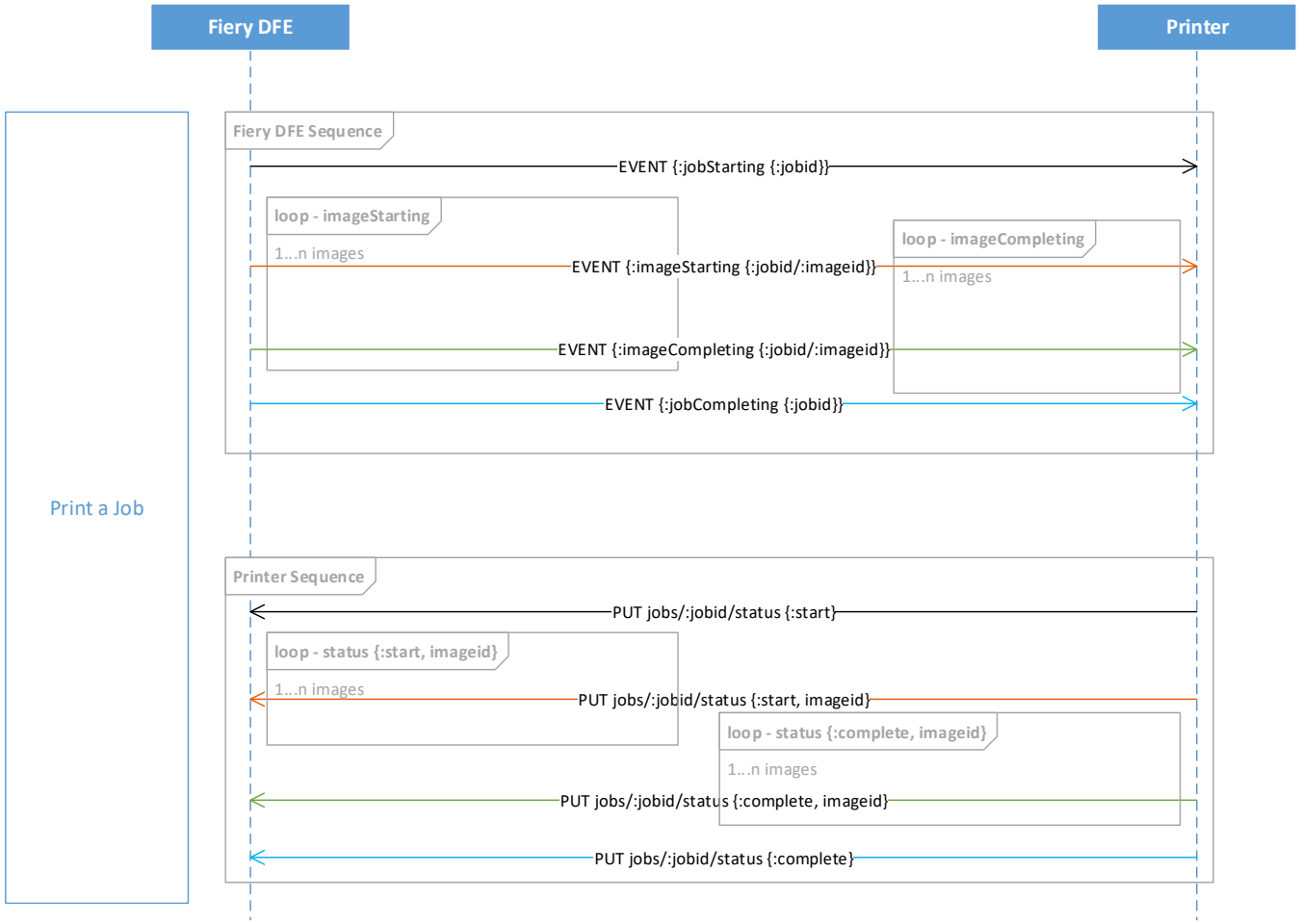


Figure 6: Print a Job Sequence

There are four different events sent by Fiery DFE during the printing sequence which require a corresponding action to be taken by the printer:

- jobStarting
- imageStarting
- imageCompleting
- jobCompleting

Aside from the **imageCompleting** event, the timing of the other three events is based solely on internal Fiery DFE events – they are not based on any action or response from the printer. The timing of the **imageCompleting** event (which is based on the time it takes to process and transfer the given image), however, can only happen after receiving a response to the corresponding **imageStarting** event. As a result, the **imageCompleting** events, although sequential in nature, can overlap the **imageStarting** and **jobCompleting** events as indicated in the sequence diagram.

The **imageCompleting** event indicates the print data for that image has been transferred to the printer. It doesn't however indicate that the printer has physically "ejected" the page. Hence, each **imageCompleting** event requires a corresponding response to indicate to Fiery DFE that the image has been printed and the resources allocated can be released on Fiery DFE. This framework is necessary to handle error conditions.

It should be noted that the response to the **jobCompleting** event is expected to be sent to Fiery DFE only after ALL **imageCompleting** events have been properly responded to by the printer.

4.2.2 Suspend/Resume

The **Suspend/Resume** sequence describes the steps a client must execute in order to temporarily suspend the Fiery DFE from sending any new **imageStarting** events. When the printer wants to initiate this action during the **Print a Job** sequence, it should set the "**dfe**" status to "**suspend**". The printer should continue to send a "**complete**" for all images which were successfully printed.

Once the printer wants to resume printing, it can set the "**dfe**" status to "**resume**". Fiery DFE will continue with the **Print a Job** sequence and resume sending **imageStarting** events, starting with the next available image. Fiery DFE will not resend any **imageStarting** events. The rest of the sequence will be the same as the **Print a Job** sequence.

To clarify using an example, if a job consists of 100 images and the "**dfe**" status was set to "**suspend**" after the **imageStarting** event for image 42, then whenever the printer sets the "**dfe**" status to "**resume**", Fiery DFE will resume the sending of **imageStarting** events, beginning with image 43 and continuing through 100.

During the time the Fiery DFE is suspended, it will continue to send **imageCompleting** events. Also, the printer can continue to send response to **imageCompleting** events. The latter is shown in the sequence diagram that follows.

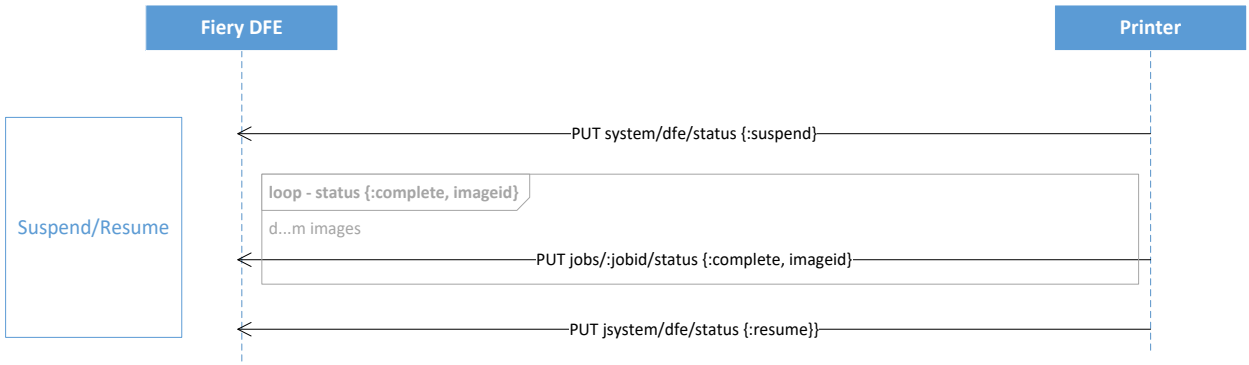


Figure 7: Suspend/Resume Sequence

4.2.3 Suspend/Recover/Resume

The **Suspend/Recover/Resume** sequence describes the steps a client must execute in order to successfully recover from an error during printing. When the printer detects an error during the **Print a Job** sequence, it should set the "**dfe**" status to "**suspend**". The printer should continue to send a "**complete**" for all images which were successfully printed. As mentioned earlier, during the time the Fiery DFE is suspended, it will continue to send **imageCompleting** events.

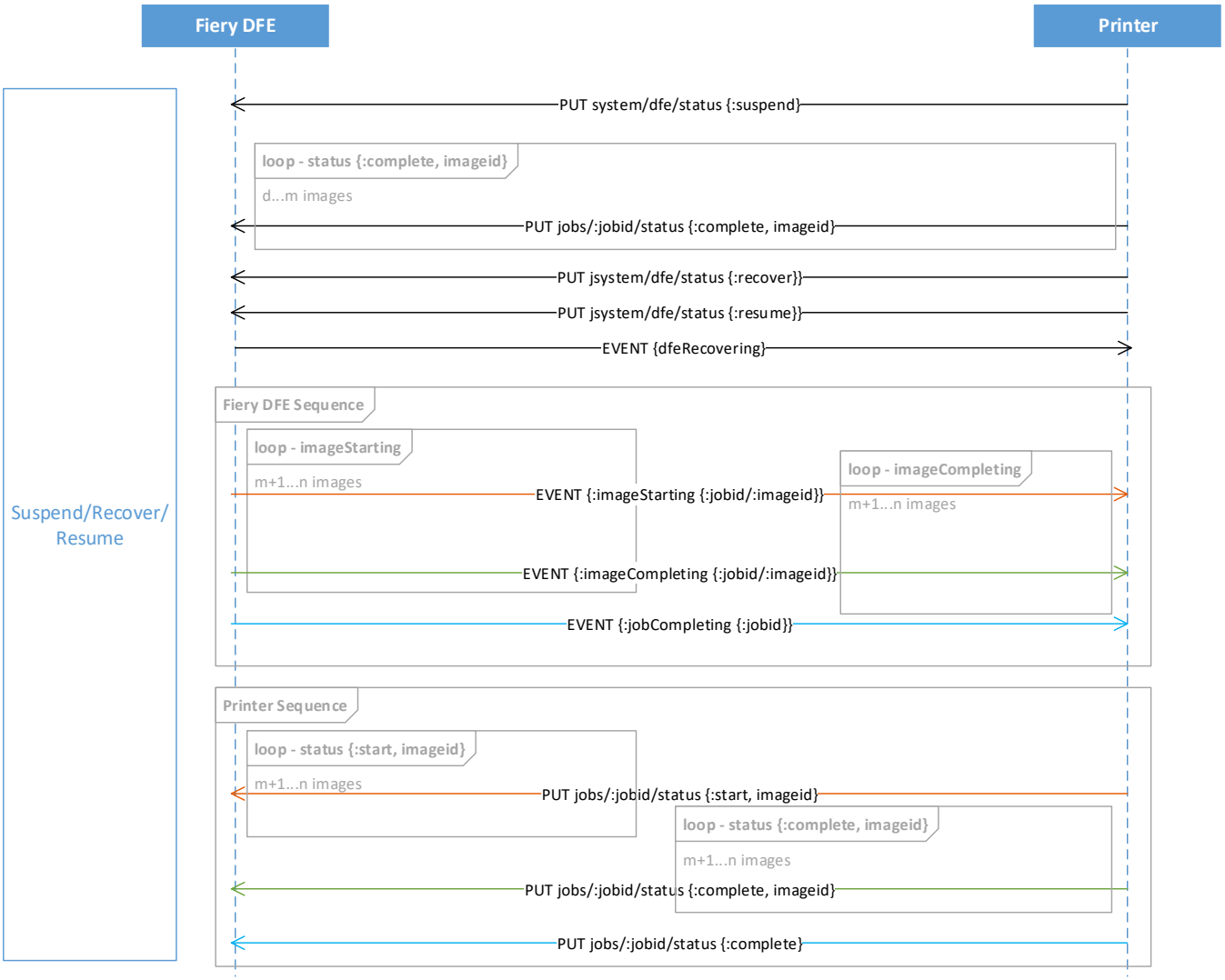


Figure 8: Suspend/Recover/Resume Sequence

If the printer plans to initiate a recovery (i.e. reprint any images that failed to print) during this period, it should set the "**dfe**" status to "**recover**". However, the printer should first send a "**complete**" for all the images for which it has received an **imageCompleting** earlier and which have been successfully printed. Once this is done for the last image, it should then send "**recover**".

Once the problem is resolved on the printer side, the printer should set the "**dfe**" status to "**resume**". Fiery DFE will then initiate the recovery process by 1) sending a **jobRecovering** event (or resending a **jobStarting** event if Fiery DFE has not received a "**complete**" status for the first image of the given job), and 2) sending the **imageStarting** events for all images of the given job which were not successfully printed (i.e. images for which Fiery DFE has not received a "**complete**" status). This will be done for each job that is part of the recovery process. This means even though Fiery DFE may have previously sent an **imageStarting** event for an image prior to the recovery process, it will send it again because a "**complete**" had not been received for that image. Once the recovery process is complete, Fiery DFE will automatically switch back to the normal printing mode and continue with the **Print a Job** sequence. The rest of the sequence will be the same as the **Print a Job** sequence, which also means the printer will receive **jobCompleting** event after the last **imageStarting** event (even though it may have received **jobCompleting** earlier).

To clarify using an example, if a job consists of 100 images and the "**dfe**" status was set to "**suspend**" after all 100 **imageStarting** events and the **jobCompleting** event were already sent, then the printer can continue to send a "**complete**" for each image that successfully prints. If only the first 25 images printed successfully (i.e. Fiery DFE receives a "**complete**" only for images 1-25), then whenever the printer sets the "**dfe**" status to "**recover**" followed by a "**resume**", Fiery DFE will send **jobRecovering** event followed by the **imageStarting** events (for images 26-100) and then resend the **jobCompleting** event.

4.2.4 Abort

The Abort sequence allows a client to successfully initiate aborting (canceling) the currently printing job, a specific job or all jobs which the printer is aware of. When the printer detects an unrecoverable error during the **Print a Job** sequence, it should set the "**dfe**" status to "**suspend**". The printer should then set the "**dfe**" status to "**abort**" (optionally with an "**id**") or "**abortall**". This will either abort (cancel) the currently printing job, the job specified by the "**id**" or in the case of "**abortall**", all jobs the printer has received a "**jobStarting**" for.

The next sequence "**Job Cancel**" describes the events and steps to successfully complete the Abort sequence. In the case of "**abortall**", the "**Job Cancel**" sequence will be repeated for each of the jobs. Once the problem has been resolved on the printer side, it is **necessary** for the printer to set the "**dfe**" status to "**resume**", otherwise printing will not resume.

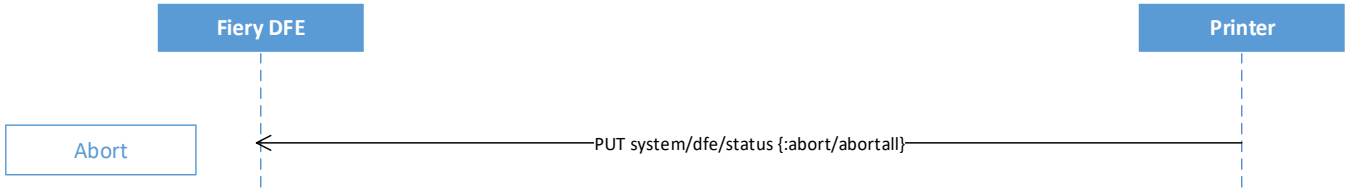


Figure 9: Abort Sequence

4.2.5 Job Cancel

The **Job Cancel** sequence describes the steps a client must execute in order to successfully complete a job cancellation. When a job is canceled, Fiery DFE will initiate a **jobCanceling** event during the "**Print a Job**" sequence. The printer should send a "**complete**" for all the images for which it has received **imageCompleting** earlier and which have been successfully printed. Once this is done for the last image, it should then send a "**cancel**" instead of "**complete**" for "**jobs**" to complete the job cancellation.

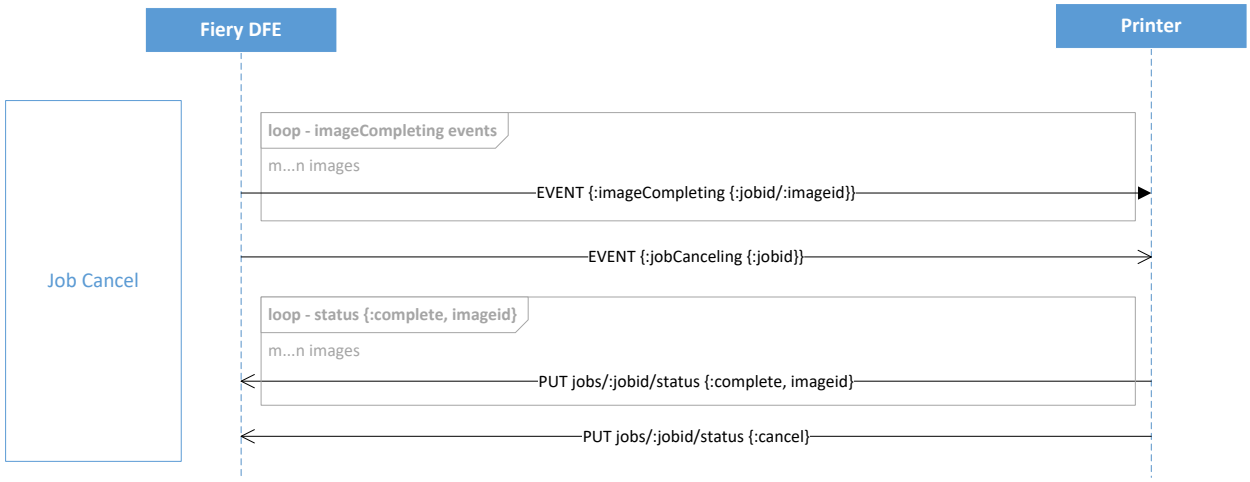


Figure 10: Job Cancel Sequence

It should be noted that if the printer has issued a "**suspend**" as part of "**Suspend/Resume**" or "**Suspend/Recover**" sequences, then it should send a "**resume**" otherwise Fiery DFE will remain in a suspended state.

4.2.6 Rush Print

The "**Rush Print"** sequence describes the steps a client must execute in order to print a rush job in the middle of a currently printing job. The sequence for it is similar to the normal **"Print a Job"** sequence with the following differences:

- The printer will receive a "**JobInterrupting**" event for the currently printing job at some point in the middle of the "**Print a Job**" sequence.
- The printer will receive a "**JobStarting**" event for the rush print job during the "**Print a Job**" sequence.
- The printer will subsequently receive "**ImageStarting**" events for all the images in the rush print job.
- The printer will receive a "**JobCompleting**" for the rush print job.
- The printer will receive a "**JobRestarting**" event for the "interrupted" print job.
- The normal "**Print a Job**" sequence will resume from this point on.

The following shows the sequence from the Fiery DFE point of view. It shows a currently printing job being interrupted by a rush print job. This is subsequently followed by the rest of the job that was interrupted. The rush print job sequence will not be interrupted. Finally, there can be more than one rush print jobs although this sequence shows only one.



Figure 11: Rush Print Fiery DFE Sequence

The following shows the sequence from the printer point of view.

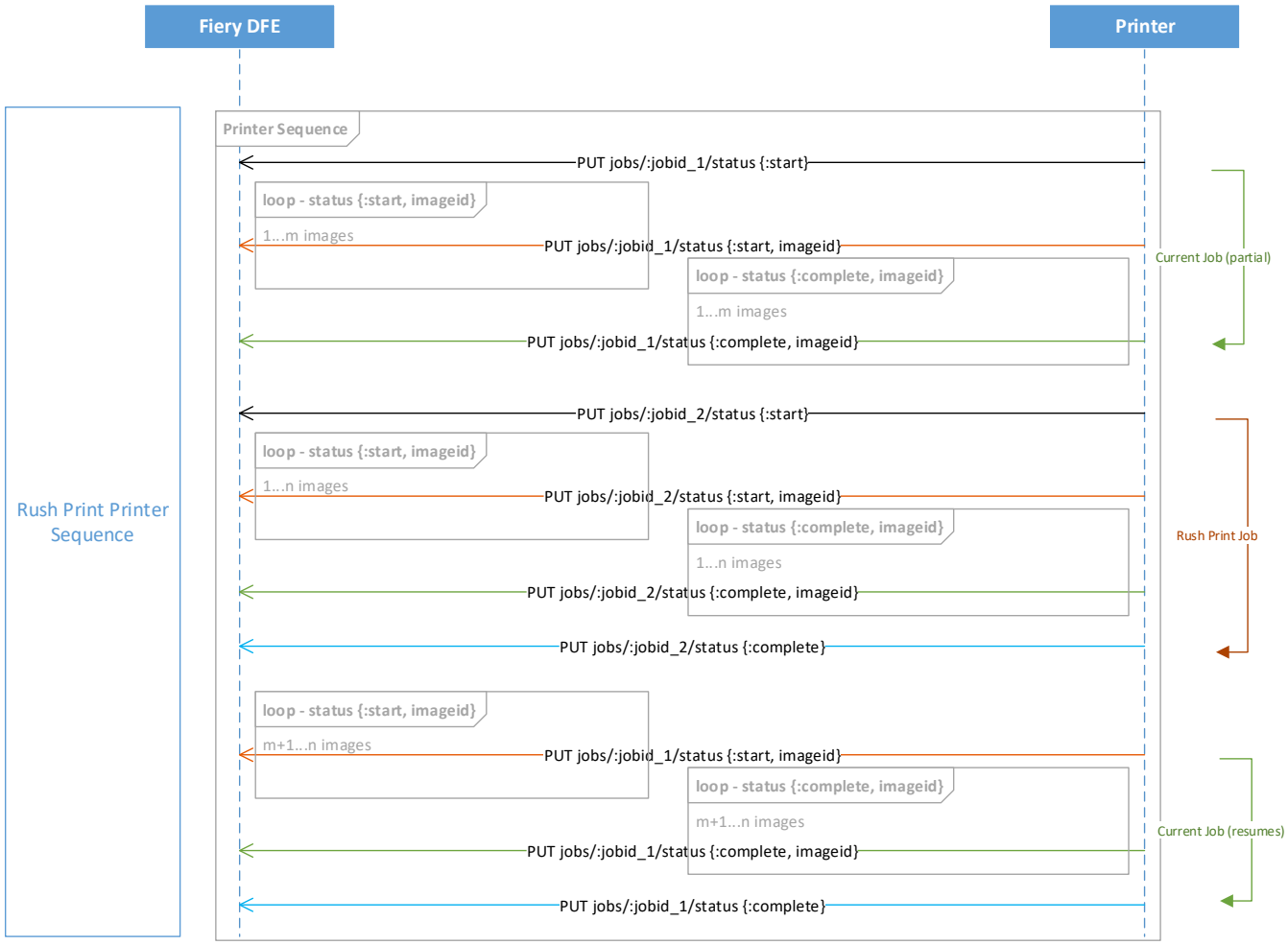


Figure 12: Rush Print Printer Sequence

4.3 Scanning

The Scanning sequences include the following sequences: Scan a Job, Scan Cancel and Scan Abort.

4.3.1 Scan a Job

The **Scan a Job** sequence describes the steps a client must execute in order to successfully scan a job. The events initiated from Fiery DFE require a corresponding action to be taken by the printer as shown in the sequence below. The Fiery DFE will respond to actions initiated by the printer as well to indicate completion of such actions.

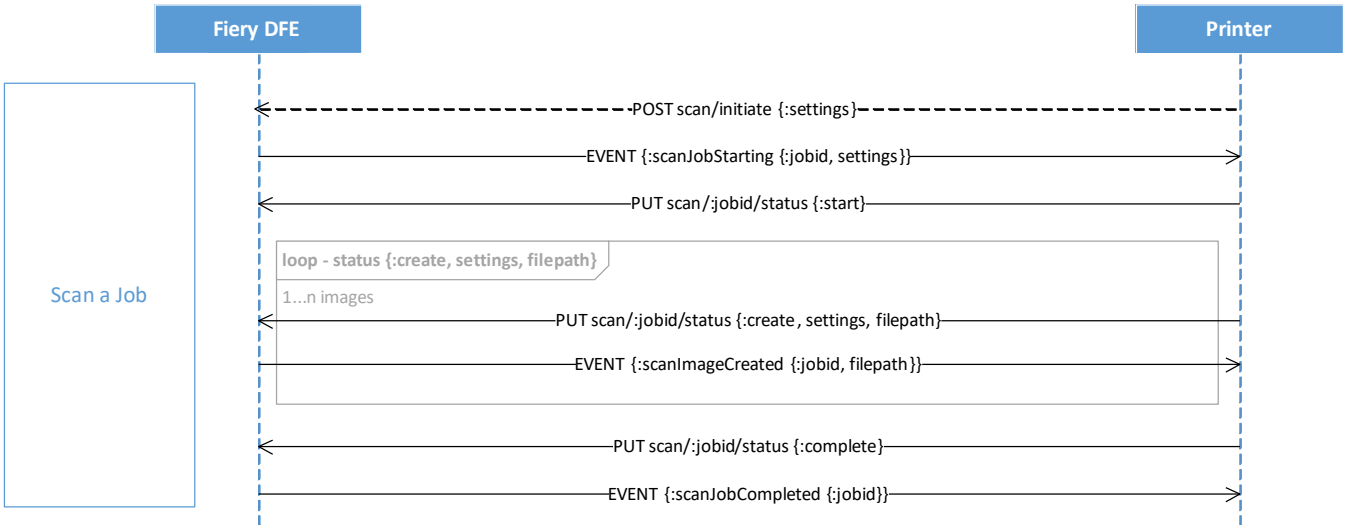


Figure 13: Scan a Job Sequence

The scan can be initiated from Fiery DFE or optionally from the printer. If it is initiated from the printer, it should provide the settings object. In either case, the printer will get **scanJobStarting** event. It will provide settings from Fiery DFE for the scan job. When the printer is ready, it should initiate a "**start**" action for the scan.

Subsequently, for each image the printer wants to add as part of this scan job, it should send a "**create**" action with settings and filepath. The filepath element will provide Fiery DFE the fully qualified path to the image. Fiery DFE will respond with **scanImageCompleted** for every image that the printer initiated a "**create**" action for.

Once the printer has added all the images for a scan, it should send a "**complete**" for the job. Fiery DFE will respond with **scanJobCompleted** when the job has been successfully added.

4.3.2 Scan Cancel

The **Scan Cancel** sequence describes the steps a client must execute in order to successfully complete a scan job cancellation. When a scan job is canceled, Fiery DFE will initiate a **scanJobCanceling** event during the "**Scan a Job**" sequence. The printer should send a "**cancel**" to complete the scan cancellation.

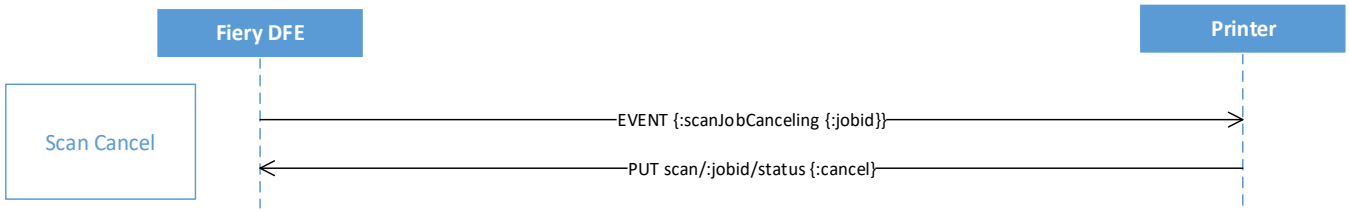


Figure 14: Scan Abort Sequence

4.3.3 Scan Abort

The **Scan Abort** sequence describes the steps a client must execute in order to successfully initiate scan cancellation. Additionally, the printer can specify a reason for the cancellation. If the printer wants to cancel a scan during "**Scan a Job**" sequence, it should send an "**abort**" action.

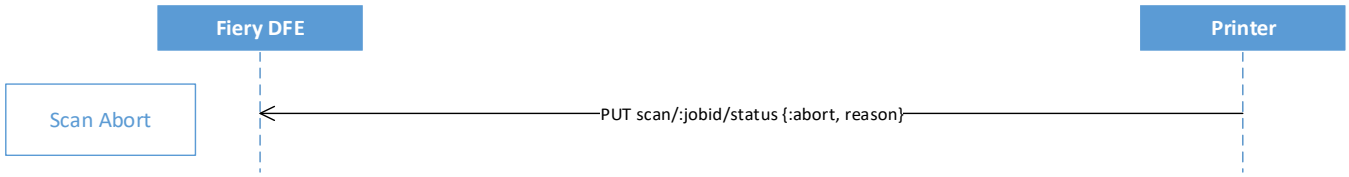


Figure 15: Scan Abort Sequence

4.4 Maintenance

The Maintenance sequences include the following: Setup Maintenance Jobs, Insert a Maintenance Job on Demand and Insert a Maintenance Job at the end of every Job.

4.4.1 Setup Maintenance Jobs

The **Setup Maintenance Jobs** sequence describes the steps a client must execute in order to successfully setup the maintenance jobs.

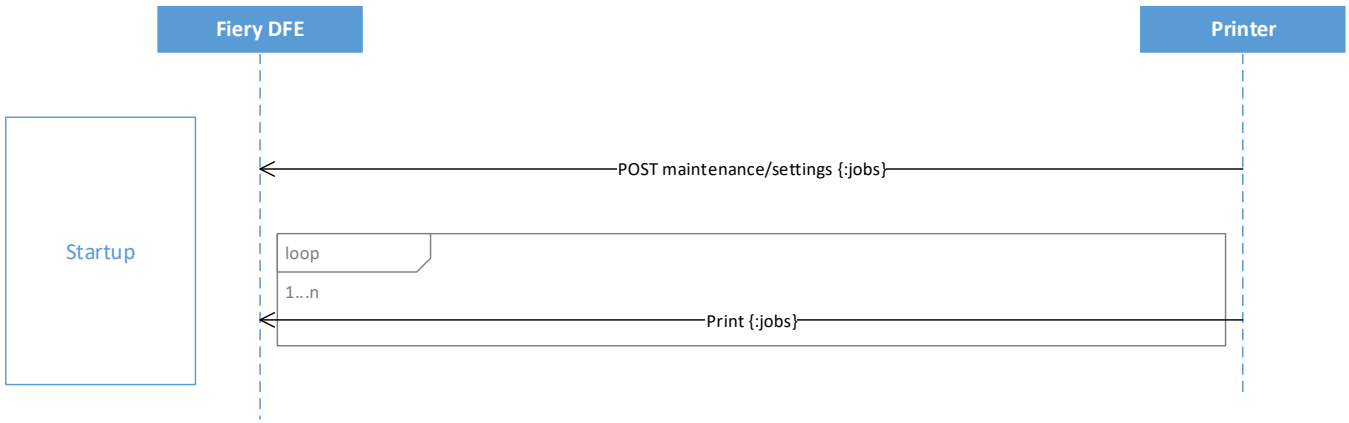


Figure 16: Setup Maintenance Jobs Sequence

At startup, the printer should send the list of maintenance jobs to the DFE. A maximum of 3 jobs can be specified in the list.

In order to successfully insert these jobs later either on demand or automatically at the end of every job, the printer must print them once at startup. When this action is initiated, the printer will get the same events as when printing a normal job.

Once the printer has done that, DFE will have everything needed to insert one of these maintenance jobs on demand in the middle of printing another job or the end of every job.

4.4.2 Insert a Maintenance Job on Demand

The **Insert a Maintenance Job on Demand** sequence describes the steps the printer must execute in order to successfully insert a maintenance job when printing another large job.

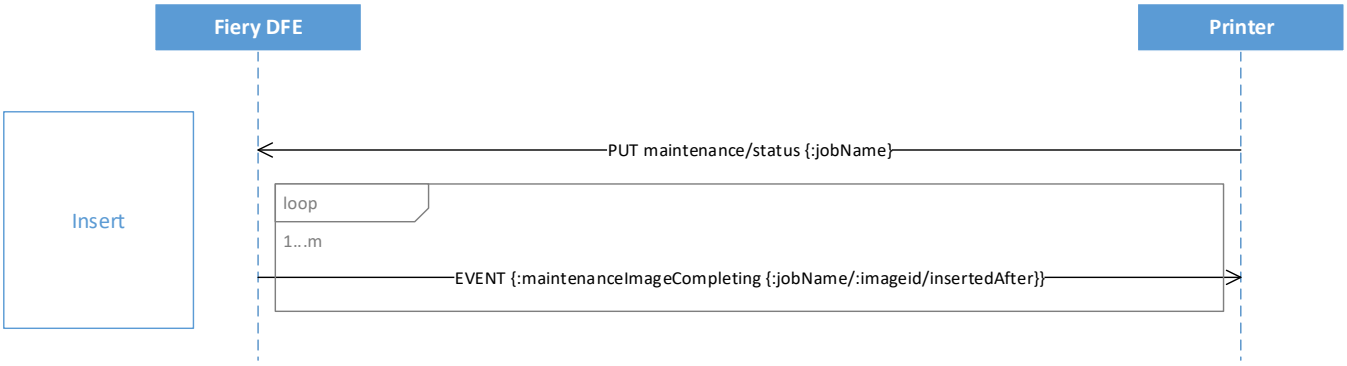


Figure 17: Insert Maintenance Job on Demand Sequence

For each of the pages in the maintenance job, **maintenanceImageCompleting** event will be sent to the printer. This event will also indicate the id of the image “after” which the maintenance images were inserted.

It is important to note that the printer can only request to insert a maintenance job that it has previously indicated to the DFE using the Setup Maintenance Jobs sequence. This includes not only indicating to the DFE the names of the maintenance jobs, but it should also have printed them once at startup.

Maintenance job can only be inserted on demand when Fiery is printing another job and is not idle. When a request for inserting a maintenance job has been made to the DFE, another request will not succeed until the previously requested maintenance job is printed.

4.4.3 Insert a Maintenance Job at the end of every Job

The **Insert a Maintenance Job at the end of every Job** sequence describes the steps the printer must execute in order to successfully insert a maintenance job at the end of every job which is printed.

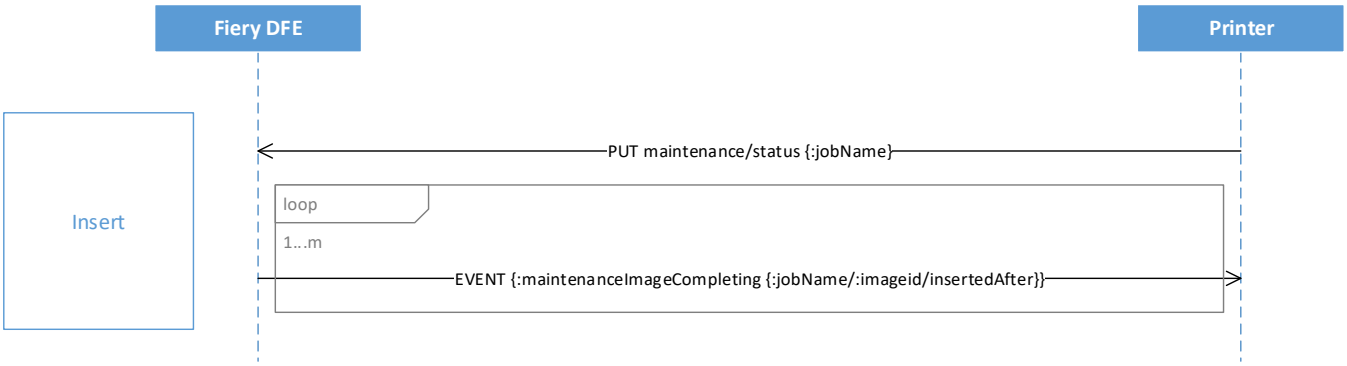


Figure 18: Insert Maintenance Job at the end of every Job Sequence

For each of the pages in the maintenance job, **maintenanceImageCompleting** event will be sent to the printer. This event will also indicate the id of the image “after” which the maintenance images were inserted.

It is important to note that the printer can only request to insert a maintenance job at the end of every printed job that it has previously indicated to the DFE using the Setup Maintenance Jobs sequence. This includes not only indicating to the DFE the names of the maintenance jobs, but it should also have printed them once at startup.

5 REST APIs

The **FieryPath** interface has an HTTPS server listening at https://<fiery_ip>fierypath/v1 supporting the various API resources mentioned below. Like any HTTPS server, the **FieryPath** interface will respond with an appropriate HTTP status code.

The **FieryPath** API is broken up into several resources. They include:

- Session
- System
- Jobs

5.1 Session

The **Session** resource contains all the sub-resources that handle session related actions. Below is a list of actions supported under the **Session** resource and how to access it:

Action	Method	Resource
Log-in	POST	https://<fiery_ip>/fierypath/v1/session/login
Log-out	POST	https://<fiery_ip>/fierypath/v1/session/logout

Table 38: Session Actions

The **Log-in** action authorizes and opens a client session. The body of the HTTPS message requires several JSON elements, as described below:

Element	Type	Required	Purpose
username	String	Yes	Name of the user attempting to log-in
password	String	Yes	Password of the user attempting to log-in
accessrights	JSON Object	Yes	JSON Object containing the “a1” element
a1	String	Yes	Access key

Table 39: Log-in Request Attributes

Here is an example of the JSON format for the body of the **Log-in** action:

```
{
    “username”: “username”,
    “password”: “password”,
    “accessrights”: {“a1”: “access_token”}
}
```

If successfully authenticated, the server will respond with the standard HTTP status code of **200** and the body will contain a valid **session_token**, as shown below:

```
{
    “token”: “token”
}
```

For the **Log-in** action, the body of the response contains the JSON elements described below:

Element	Type	Required	Purpose
token	String	Yes	Token to be used for the session

Table 40: Log-in Response Attributes

For the **Log-in** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check credentials provided
503	Service Unavailable	Error – the service is unavailable temporarily, try again

Table 41: Log-in HTTP Status Codes

The **Log-in** action should be the first resource accessed by a client because the returned **token** is required to access all other FieryPath API resources. To properly access subsequent **FieryPath** API resources, this **token** needs to be provided in the HTTPS header in the format:

Authorization: Bearer **token**

Note that during a startup phase as in when the Fiery DFE machine is restarted, the client may need to initiate the **Log-in** sequence more than once until it succeeds (which will happen once Fiery DFE is fully initialized). Once a client successfully performs the **Log-in** action, the first **mandatory** step is to open a WebSocket connection. The FieryPath interface has a WebSocket server listening at wss://<fiery_ip>/fierypathevents/v1. This WebSocket server is used to send Fiery DFE events as described in the earlier sections. In order to authenticate, clients need to send a "*connect*" message from a WebSocket client in following JSON format in the body of the request:

```
{
    "jsonrpc": "2.0",
    "method": "connect",
    "params": {"token": "token"}
}
```

The **session_token** is obtained from the response to the **Log-in** action as indicated above. If successfully connected, the WebSocket server will respond with the following JSON object:

```
{
    "jsonrpc": "2.0",
    "result": "auth success"
}
```

Just like the **Log-in** sequence, the client may need to initiate the WebSocket connection more than once until it is successful.

The **Log-out** action closes a client session. When a client accesses this resource with a valid **session_token**, the session is closed, and further use of this token is no longer allowed. A client does not have to separately close the WebSocket connection. It is automatically closed as part of the **Log-out** action.

For the **Log-out** action, the body of the response contains the JSON elements described below:

Element	Type	Required	Purpose
authenticated	Boolean	Yes	Set to false indicating a logout

Table 42: Log-out Response Attributes

For the **Log-out** action, the valid HTTP status codes are as follows:

Code	Indication	Description
201	OK	Success
401	Unauthorized	Error - check for successful log-in

Table 43: Log-out HTTP Status Codes

5.2 System

The **System** resource contains 2 sub-resources (DFE and printer) that handle configuration/settings related actions. For the DFE, this currently includes: Status. For the printer sub-resource, this currently includes: Status, Inks, Inputs, Localization, OEM Configuration, OEM Corrections, and Finishers.

5.2.1 Status (DFE)

The **Status** sub-resource contains everything needed to manage the status of the DFE. Below is a list of actions supported under the status sub-resource and how to access it:

Action	Method	URL
Get a List of DFE Statuses	GET	https://<fiery_ip>/fierypath/v1/system/dfe/status
Set DFE Status	PUT	https://<fiery_ip>/fierypath/v1/system/dfe/status

Table 44: DFE Status Actions

The **Get a List of DFE Statuses** action retrieves the current list of DFE status entries. The body of the response contains the JSON elements described below:

Element	Type	Required	Purpose
state	String	Yes	Valid values include: “connected”, “disconnected” or “ready”
warnings	JSON Array	No	Array of Strings

Table 45: Get DFE Status Response Attributes

For the **Get DFE Status** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in

Table 46: Get DFE Status HTTP Status Codes

The **Set DFE Status** action allows suspending, resuming/recovering, restarting/rebooting the DFE. In addition, the currently printing or a specific job can be canceled by setting the action to "**abort**" or all jobs by using "**abortall**". The body of the HTTPS message requires a single JSON object, containing the JSON elements described below. In the case of "**abort**" an optional "**reason**" can be specified to indicate why the "**abort**" was issued. Additionally, an "**id**" can be specified to "**abort**" a specific job. For "**abortall**", an optional "**reason**" can be used as well. If the printer wants to send either "**abort**" or "**abortall**" without requiring that it should respond to the corresponding "**jobCanceling**" event, it can also specify an optional "**responseRequired**" attribute and set it to false. In the case of "**recover**" an optional "**id**" can be specified to indicate where the printer wants printing to restart from. This may not correspond to the first image after the last image which the printer responded with "**image complete**" (due to the behavior of various finishers). For those images after the last "**image complete**", but prior to the image specified by "**id**", the printer is expected to eventually issue an "**image complete**" for each of those images, unless the optional "**responseRequired**" attribute is included and set to false. Finally, if the action is set to "**restart**", the DFE will be restarted and correspondingly, if the action is set to "**reboot**", the DFE will be rebooted.

Element	Type	Required	Purpose
action	String	Yes	Valid values include: "suspend", "resume", "recover", "abort", "abortall", "restart", "reboot"
reason	String	No	Indicates the reason why "abort", "abortall", or "suspend" was issued
id	String	No	If "action" is "abort", indicates the "job" to abort. If "action" is "recover", indicates the "image" where the printer wants to recover printing from (once a "resume" is issued). If "action" is "suspend", indicates the "job" to suspend.
responseRequired	Boolean	No	If "action" is "abort" or "abortall", indicates if printer will respond to "jobCanceling" event(s). If "action" is "recover", indicates if printer will respond to all "imageCompleting" events issued prior to the "recover" for "images" that came before the specified "id" (once "resume" is issued). Valid only when a valid "id" is specified.

Table 47: Set DFE Status Request Attributes

For the **Set DFE Status** action, the body of the response contains the JSON elements described below:

Element	Type	Required	Purpose
state	String	Yes	Valid values include: "connected" or "ready"
warnings	JSON Array	No	Array of Strings

Table 48: Set DFE Status Response Attributes

For the **Set DFE Status** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
422	Unprocessable Entity	Error – invalid configuration parameter. Check "Details" for more information

Table 49: Set DFE Status HTTP Status Codes

5.2.2 Status (Printer)

The **Status** sub-resource contains everything needed to manage the status of the printer. Below is a list of actions supported under the status sub-resource and how to access it. Note that printer status will be defined by the printer in general. However, once the printer status is set to “ready”, DFE will transition to “ready” state as indicated in the **Start-up** sequence diagram.

Action	Method	URL
Get the Printer Status	GET	https://<fiery_ip>/fierypath/v1/system/printer/status
Set the Printer Status	PUT	https://<fiery_ip>/fierypath/v1/system/printer/status

Table 50: Printer Status Actions

The **Get the Printer Status** action retrieves current printer status. The body of the response contains these JSON elements:

Element	Type	Required	Purpose
state	String	Yes	Valid values include: “initializing”, “connected” or “ready”
info	JSON Array	No	Information messages of type “String” sent by the printer.
warnings	JSON Array	No	Warning messages of type “String” sent by the printer.
errors	JSON Array	No	Error messages of type “String” sent by the printer.

Table 51: Get Printer Status Response Attributes

For the **Get Printer Status** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in

Table 52: Get Printer Status HTTP Status Codes

The **Set the Printer Status** action updates the printer status. This is defined by the printer. The body of the HTTPS message requires a single JSON object, containing the JSON elements described below:

Element	Type	Required	Purpose
action	String	Yes	Valid values include: “initializing”, “connected” or “ready”
info	JSON Array	No	Information messages of type “String”.
warnings	JSON Array	No	Warning messages of type “String.
errors	JSON Array	No	Error messages of type “String.

Table 53: Set Printer Status Request Attributes

At startup, the printer can set its status to “**initializing**”. This is optional and should be done only once. Subsequently, until the printer has sent all configurations, it can use the “**connected**” status. Once the printer is ready, it should set the “**ready**” status. In order to receive printing events, it is **mandatory** that the printer set its status to “**ready**” after setting initial configuration.

For info, warnings and errors, although they are of type “JSON Array”, **only the first message will be displayed**. The type “JSON Array” for these elements is for future enhancements. The printer should currently provide one element array for each when it wants to set a message.

It is **important** to note that the full set needs to be sent by the printer for all the info, warnings and errors messages. As an example, if a “warnings” message is sent and then at some point later an “errors” message is sent, only the “errors” message will be displayed unless the JSON object contains both “warnings” and “errors”. Localized messages will be displayed in appropriate applications if the printer has sent localization information to the DFE. For more information on localization, see 5.2.6.

In order to clear messages, the printer should send a zero-element array for the type of message that needs to be cleared.

For the **Set Printer Status** action, the body of the response contains the JSON elements described below:

Element	Type	Required	Purpose
state	String	Yes	Valid values include: “initializing”, “connected” or “ready”
info	JSON Array	No	Information messages of type “String” sent by the printer.
warnings	JSON Array	No	Warning messages of type “String” sent by the printer.
errors	JSON Array	No	Error messages of type “String” sent by the printer.

Table 54: Set Printer Status Response Attributes

For the **Set Printer Status** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
422	Unprocessable Entity	Error – invalid configuration parameter. Check “Details” for more information

Table 55: Set Printer Status HTTP Status Codes

5.2.3 OEM Configuration (Printer)

The **OEM Configuration** sub-resource contains everything needed to manage printer specific configuration outside of inks, inputs and localization. This provides the capability for printers to send OEM specific JSON BLOB to DFE. The definition and contents of this BLOB is based on an agreement between the printer and DFE. Below is a list of actions supported under the **OEM Configuration** sub-resource and how to access it:

Action	Method	URL
Get Configuration	GET	https://<fiery_ip>/fierypath/v1/system/printer/oem/configuration
Create Configuration	POST	https://<fiery_ip>/fierypath/v1/system/printer/oem/configuration
Delete Configuration	DELETE	https://<fiery_ip>/fierypath/v1/system/printer/oem/configuration

Table 56: Configuration Actions

The **Get Configuration** action will indicate if the printer has set the configuration. The response body will indicate this with the contents being the JSON configuration set by the printer.

For the **Get Configuration** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
204	No Content	Error – no OEM configuration resource available
401	Unauthorized	Error – check for successful log-in

Table 57: Get Configuration HTTP Status Codes

The **Create Configuration** action creates a new configuration sub-resource. The body of the HTTPS message requires a single JSON object containing a printer specific JSON BLOB.

Element	Type	Required	Purpose
configuration	JSON Object	Yes	Contains a printer specific JSON BLOB

Table 58: Create Configuration Request Attributes

Here is an example of the JSON object for the **Create Configuration** request:

```
{
    "configuration": {
        "BLOB": {"Data": "printer specific"}
    }
}
```

For the **Create Configuration** action, the valid HTTP status codes are as follows:

Code	Indication	Description
201	OK	Success
401	Unauthorized	Error – check for successful log-in
422	Unprocessable Entity	Error – invalid configuration parameter. Check “Details” for more information
429	Too many requests	Error – operation already in progress

Table 59: Create Configuration HTTP Status Codes

The **Delete Configuration** action removes the configuration sub-resource created earlier. The valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – OEM configuration resource is not created

Table 60: Delete Configuration HTTP Status Codes

5.2.4 Inks (Printer)

The **Inks** sub-resource contains everything needed to manage the list of inks. Below is a list of actions supported under the **Inks** sub-resource and how to access it:

Action	Method	URL
Get a List of Installed Inks	GET	https://<fiery_ip>/fierypath/v1/system/printer/inks
Create an Ink	POST	https://<fiery_ip>/fierypath/v1/system/printer/inks
Delete an Ink	DELETE	https://<fiery_ip>/fierypath/v1/system/printer/inks/<id>
Delete all Inks	DELETE	https://<fiery_ip>/fierypath/v1/system/printer/inks
Get an Ink’s Attributes	GET	https://<fiery_ip>/fierypath/v1/system/printer/inks/<id>
Set an Ink’s Attributes	PUT	https://<fiery_ip>/fierypath/v1/system/printer/inks/<id>
Get an Ink’s Status	GET	https://<fiery_ip>/fierypath/v1/system/printer/inks/<id>/status
Set an Ink’s Status	PUT	https://<fiery_ip>/fierypath/v1/system/printer/inks/<id>/status

Table 61: Ink Actions

The **Get a List of Installed Inks** action retrieves the current list of ink entries. The response body will contain a JSON array (based on the number of installed inks), containing the JSON elements described below:

Element	Type	Required	Purpose
id	Number	Yes	Indicates the ID of the ink
type	String	Yes	Typical values are “cyan”, “magenta”, “yellow”, “black”. The full list of supported inks is product specific
capacity	Number	Yes	Indicates the total capacity of the ink
dropsize	JSON Array	No	Array of drop sizes in picoliters
downsample	JSON Object	No	Object indicating the downsample resolution

Table 62: Get a List of Installed Inks Response Attributes

If successful, **id** will be a number (which is required for accessing other resources related to this ink) valid only for the duration of the session. The **capacity** attribute is used to indicate the maximum amount of ink that can be installed. The value is expressed in whatever unit the printer supports (e.g. percentage, liters, gallons, etc.). The optional **dropsize** attribute indicates the drop size in picoliters. The optional **downsample** attribute indicates the resolution at which the ink can be downsampled. Both **dropsize** and **downsample** will be available only if the printer has created the ink with this information.

For the **Get a List of Installed Inks** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
204	No Content	Error – no installed ink
401	Unauthorized	Error – check for successful log-in

Table 63: Get a List of Installed Inks HTTP Status Codes

The **Create an Ink** action creates a new ink and adds it to the list of installed inks. The body of the HTTPS message requires a single **ink** JSON object, containing the couple of JSON elements described below:

Element	Type	Required	Purpose
type	String	Yes	Typical values are “cyan”, “magenta”, “yellow”, “black”. The full list of supported inks is product specific
capacity	Number	Yes	Indicates the total capacity of the ink
dropsize	JSON Array	No	Array of drop sizes in picoliters
downsample	JSON Object	No	Object indicating the downsample resolution

Table 64: Create an Ink Request Attributes

Here is an example of the JSON object for the **Create an Ink** request:

```
{
    "ink": {
        "type": "cyan",
        "capacity": 100,
        “dropsize”: [3, 6, 9, 12, 15, 18],
        “downsample”: {“x”: 360, “y”: 360}
    }
}
```

For the **Create an Ink** action, the body of the response contains the JSON elements described below:

Element	Type	Required	Purpose
id	Number	Yes	Indicates the ID of the ink

Table 65: Create an Ink Response Attributes

For the **Create an Ink** action, the valid HTTP status codes are as follows:

Code	Indication	Description
201	OK	Success
401	Unauthorized	Error – check for successful log-in
422	Unprocessable Entity	Error – invalid configuration parameter. Check “Details” for more information
503	Service Unavailable	Error – the service is unavailable temporarily, try again

Table 66: Create an Ink HTTP Status Codes

The **Delete an Ink** action removes the given ink from the list of installed inks. The valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – ink with specified id is not installed

Table 67: Delete an Ink HTTP Status Codes

The **Delete all Inks** action removes all the installed inks. The valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – no installed inks

Table 68: Delete all Inks HTTP Status Codes

The **Get an Ink’s Attributes** action retrieves the current list of attributes for the given ink. The body of the response contains the JSON elements described below:

Element	Type	Required	Purpose
id	Number	Yes	Indicates the ID of the ink
type	String	Yes	Typical values are “cyan”, “magenta”, “yellow”, “black”. The full list of supported inks is product specific
capacity	Number	Yes	Indicates the total capacity of the ink
dropsize	JSON Array	No	Array of drop sizes in picoliters
downsample	JSON Object	No	Object indicating the downsample resolution

Table 69: Get an Ink's Attributes Response Attributes

For the **Get an Ink's Attributes** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – ink with specified id is not installed

Table 70: Get an Ink's Attributes Response HTTP Status Codes

The **Set an Ink's Attributes** action updates the current list of attributes for the given ink. The body of the HTTPS message requires a single **ink** JSON object, containing the JSON elements described below:

Element	Type	Required	Purpose
capacity	Number	Yes	Indicates the total capacity of the ink

Table 71: Set an Ink's Attributes Request Attributes

Here is an example of the JSON object for **Set an Ink's Attributes** request:

```
{
    "ink": {"capacity": 100}
}
```

For the **Set an Ink's Attributes** action, the body of the response contains the JSON elements described below:

Element	Type	Required	Purpose
id	Number	Yes	Indicates the ID of the ink
type	String	Yes	Typical values are "cyan", "magenta", "yellow", "black". The full list of supported inks is product specific
capacity	Number	Yes	Indicates the total capacity of the ink
dropsize	JSON Array	No	Array of drop sizes in picoliters
downsample	JSON Object	No	Object indicating the downsample resolution

Table 72: Set an Ink's Attributes Response Attributes

For the **Set an Ink's Attributes** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – ink with specified id is not installed
422	Unprocessable Entity	Error – invalid configuration parameter. Check "Details" for more information

Table 73: Set an Ink's Attributes HTTP Status Codes

The **Get an Ink's Status** action retrieves the current status for the given ink. The body of the response contains the JSON elements described below:

Element	Type	Required	Purpose
level	Number	Yes	Indicates the current level/amount of ink
state	String	Yes	Indicates if the ink is enabled or disabled

Table 74: Get an Ink's Status Response Attributes

The **level** attribute is used to indicate the current amount of ink installed. The value is expressed in the same unit as the

capacity attribute in and must be less than or equal to the **capacity** attribute's value.

For the **Get an Ink's Status** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – ink with specified id is not installed

Table 75: Get an Ink's Status HTTP Status Codes

The **Set an Ink's Status** action updates the current status for the given ink. The body of the HTTPS message requires a single **ink** JSON object, containing the JSON elements described below:

Element	Type	Required	Purpose
level	Number	Yes	Indicates the current level/amount of ink
state	String	No	Valid values are "enabled" or "disabled". If not specified, the default is enabled

Table 76: Set an Ink's Status Request Attributes

Here is an example of the JSON object for Set **an Ink's** Status request:

```
{
    "ink": {"level": 50, "state": "enabled"}
}
```

For the **Set an Ink's Status** action, the body of the response contains the JSON elements described below:

Element	Type	Required	Purpose
level	Number	Yes	Indicates the current level/amount of ink
state	String	Yes	Indicates if the ink is enabled or disabled

Table 77: Set an Ink's Status Response Attributes

For the **Set an Ink's Status** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
403	Forbidden	Error – ink level cannot be greater than capacity
404	Not Found	Error – ink with specified id is not installed
422	Unprocessable Entity	Error – invalid configuration parameter. Check "Details" for more information

Table 78: Set an Ink's Status HTTP Status Codes

5.2.5 Inputs (Printer)

The **Inputs** sub-resource contains everything needed to manage the list of media inputs/sources. Below is a list of actions supported under the **Inputs** sub-resource and how to access it:

Action	Method	URL
Get a List of Installed Inputs	GET	https://<fiery_ip>/fierypath/v1/system/printer/inputs
Create an Input	POST	https://<fiery_ip>/fierypath/v1/system/printer/inputs
Delete an Input	DELETE	https://<fiery_ip>/fierypath/v1/system/printer/inputs/<id>
Delete all Inputs	DELETE	https://<fiery_ip>/fierypath/v1/system/printer/inputs
Get an Input's Attributes	GET	https://<fiery_ip>/fierypath/v1/system/printer/inputs/<id>
Set an Input's Attributes	PUT	https://<fiery_ip>/fierypath/v1/system/printer/inputs/<id>
Get an Input's Status	GET	https://<fiery_ip>/fierypath/v1/system/printer/inputs/<id>/status
Set an Input's Status	PUT	https://<fiery_ip>/fierypath/v1/system/printer/inputs/<id>/status

Table 79: Input Actions

The **Get a List of Installed Inputs** action retrieves the current list of installed inputs. The response body will contain a JSON array (based on the number of installed inputs), containing the JSON elements described below:

Element	Type	Required	Purpose
id	Number	Yes	Indicates the ID of the input
name	String	Yes	Indicates the name of the input
number	Number	No	Refer to product specific documentation.
capacity	Number	Yes	Indicates the total capacity of the input.
autoSelect	Boolean	No	Indicates whether or not the input is available for auto-select (default = false).

Table 80: Get a List of Installed Inputs Response Attributes

If successful, **id** will be a number (which is required for accessing other resources related to this input) valid only for the duration of the session. The **capacity** attribute is used to indicate the maximum amount of substrate/media/paper that can be held. The value is expressed in whatever unit the printer supports (e.g. percentage, sheets, meters, yards, etc.). The **autoSelect** attribute determines whether or not this input can be used by the DFE when trying to fulfill a user's **autoSelect** request for selecting an input when printing a job.

For the **Get a List of Installed Inputs** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
204	No Content	Error – no installed input
401	Unauthorized	Error – check for successful log-in

Table 81: Get a List of Installed Inputs HTTP Status Codes

The **Create an Input** action creates a new input and adds it to the list of installed inputs. The body of the HTTPS message requires a single **input** JSON object, containing the several JSON elements described below:

Element	Type	Required	Purpose
name	String	Yes	Indicates the name of the input.
number	Number	No	Refer to product specific documentation.
capacity	Number	Yes	Indicates the total capacity of the input.
autoSelect	Boolean	No	Indicates whether or not the input is available for auto-select (default = false).

Table 82: Create an Input Request Attributes

IHere is an example of the JSON object for **Create an Input** request:

```
{
    "input": {
        "name": "Tray1",
        "capacity": 5000,
        "autoSelect": true
    }
}
```

For the **Create an Input** action, the body of the response contains the JSON elements described below:

Element	Type	Required	Purpose
id	Number	Yes	Indicates the ID of the input

Table 83: Create an Input Response Attributes

For the **Create an Input** action, the valid HTTP status codes are as follows:

Code	Indication	Description
201	OK	Success
401	Unauthorized	Error – check for successful log-in
422	Unprocessable Entity	Error – invalid configuration parameter. Check “Details” for more information

Table 84: Create an Input HTTP Status Codes

The **Delete an Input** action removes the given input from the list of installed inputs. The valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – input with specified id is not installed

Table 85: Delete an Input HTTP Status Codes

The **Delete all Inputs** action removes all the installed inputs. The valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – no installed inputs

Table 86: Delete all Inputs HTTP Status Codes

The **Get an Input's Attributes** action retrieves the current list of attributes for the given input. The body of the response contains the JSON elements described below:

Element	Type	Required	Purpose
id	Number	Yes	Indicates the ID of the input.
name	String	Yes	Indicates the name of the input.
number	Number	No	Refer to product specific documentation.
capacity	Number	Yes	Indicates the total capacity of the input.
autoSelect	Boolean	No	Indicates whether or not the input is available for auto-select (default = false).

Table 87: Get an Input's Attributes Response Attributes

For the **Get an Input's Attributes** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – input with specified id is not installed

Table 88: Get an Input's Attributes HTTP Status Codes

The **Set an Input's Attributes** action updates the current list of attributes for the given input. The body of the HTTPS message requires a single **input** JSON object, containing the several JSON elements described below:

Element	Type	Required	Purpose
name	String	Yes	Indicates the name of the input.
number	Number	No	Refer to product specific documentation.
capacity	Number	Yes	Indicates the total capacity of the input.
autoSelect	Boolean	No	Indicates whether or not the input is available for auto-select (default = false).

Table 89: Set an Input's Attributes Request Attributes

Here is an example of the JSON object for **Set an Input's Attributes** request:

```
{
    "input": {
        "name": "Tray1",
        "capacity": 5000,
        "autoSelect": true
    }
}
```

For the **Set an Input's Attributes** action, the body of the response contains the JSON elements described below:

Element	Type	Required	Purpose
id	Number	Yes	Indicates the ID of the input.
name	String	Yes	Indicates the name of the input.
number	Number	No	Refer to product specific documentation.
capacity	Number	Yes	Indicates the total capacity of the input.
autoSelect	Boolean	No	Indicates whether or not the input is available for auto-select (default = false).

Table 90: Set an Input's Attributes Response Attributes

For the **Set an Input's Attributes** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – input with specified id is not installed
422	Unprocessable Entity	Error – invalid configuration parameter. Check "Details" for more information

Table 91: Set an Input's Attributes HTTP Status Codes

The **Get an Input's Status** action retrieves the current status for the given input. The body of the response contains the JSON elements described below:

Element	Type	Required	Purpose
state	String	Yes	Indicates the status of the input. Valid "state" values include: "ready", "disabled"
level	Number	Yes	Indicates the current level/amount of media in the input.
media	JSON Object	Yes	JSON Object containing the "media" elements. Refer to Set an Input's Status
sizeInPoints	JSON Object	No	JSON Object containing "x" and "y" elements.
x	Number	No	Indicates size in points of the media's width
y	Number	No	Indicates size in points of the media's height
size	String	No	Indicates Page Size/Media Name of the media in the input. For eg: "Letter", "A4", etc.
color	String	No	Indicates Media Color of media in the input.
type	String	No	Indicates Media Type of media in the input. For eg: "Plain", "Recycled", etc.
weight	String	No	Indicates Media Weight of the media in the input.
dimUnit	String	No	Indicates Dimension Unit of media in the input. Valid values are "micrometers", "tenThousandthsOfInches", "unknown" and "other".
capacityUnit	String	No	Indicates Tray Capacity Unit. Valid values are "tenThousandthsOfInches", "micrometers", "sheets", "feet" and "meters".
duplex	Boolean	No	Indicates whether the Tray supports Duplex – (default is false)
trayType	String	No	Indicates type of the Tray. Valid values are "sheetFeedAutoRemovableTray", "sheetFeedAutoNonRemovableTray", "sheetFeedManual", "continuousRoll", "continuousFanFold", "other", "unknown".
security	String	No	Indicates Tray security options. Valid values are "on", "off", "notPresent", "other".
description	String	No	Indicates string value describing Tray.
vendorName	String	No	Indicates Vendor name for the Press/Tray
serialNumber	String	No	Indicates Serial Number information for the Tray.

Table 92: Get an Input's Status Response Attributes

The **level** attribute is used to indicate the current amount of media installed. The value is expressed in the same unit as the **capacity** attribute in and must be less than or equal to the **capacity** attribute's value. The **sizeInPoints** attribute is the dimensions of the substrate/media/paper that is loaded in the input.

For the **Get an Input's Status** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
403	Forbidden	Error – input level not set
404	Not Found	Error – input with specified id is not installed

Table 93: Get an Input's Status HTTP Status Codes

The **Set an Input's Status** action updates the status for the given input and the information of print media that it holds.

The body of the HTTPS message requires a single **input** JSON object, containing the JSON element described below:

Element	Type	Required	Purpose
state	String	Yes	Indicates the status of the input. Valid "state" values include: "ready", "disabled"
level	Number	Yes	Indicates the current level/amount of media in the input.
media	JSON Object	Yes	JSON Object containing the "media" elements. Size of media if available should be reported through sizeInPoints. Product may support additional characteristics of media such as type, weight, etc. These should also be reported under media object. Refer to product documentation for list of elements that goes under media object.
sizeInPoints	JSON Object	No	JSON Object containing "x" and "y" elements.
x	Number	No	Indicates size in points of the media's width
y	Number	No	Indicates size in points of the media's height
size	String	No	Indicates Page Size/Media Name of the media in the input. For eg: "Letter", "A4", etc.
color	String	No	Indicates Media Color of media in the input.
type	String	No	Indicates Media Type of media in the input. For eg: "Plain", "Recycled", etc.
weight	String	No	Indicates Media Weight of the media in the input.
dimUnit	String	No	Indicates Dimension Unit of media in the input. Valid values are "micrometers", "tenThousandthsOfInches", "unknown" and "other".
capacityUnit	String	No	Indicates Tray Capacity Unit. Valid values are "tenThousandthsOfInches", "micrometers", "sheets", "feet" and "meters".
duplex	Boolean	No	Indicates whether the Tray supports Duplex – (default is false)
trayType	String	No	Indicates type of the Tray. Valid values are "sheetFeedAutoRemovableTray", "sheetFeedAutoNonRemovableTray", "sheetFeedManual", "continuousRoll", "continuousFanFold", "other", "unknown".
security	String	No	Indicates Tray security options. Valid values are "on", "off", "notPresent", "other".
description	String	No	Indicates string value describing Tray.
vendorName	String	No	Indicates Vendor name for the Press/Tray
serialNumber	String	No	Indicates Serial Number information for the Tray.

Table 94: Set an Input's Status Request Attributes

Here is an example of the JSON object for **Set an Input's Status** request:

```
{
    "input": {
        “state”: “ready”,
        "level": 50,
        “media: { … }
    }
}
```

For the **Set an Input's Status** action, the body of the response contains the JSON elements described below:

Element	Type	Required	Purpose
state	String	Yes	Indicates the status of the input. Valid “state” values include: “ready”, “disabled”
level	Number	Yes	Indicates the current level/amount of media in the input.
media	JSON Object	Yes	JSON Object containing the “media” elements. Refer to Set an Input's Status..
sizeInPoints	JSON Object	No	JSON Object containing “x” and “y” elements.
x	Number	No	Indicates size in points of the media's width
y	Number	No	Indicates size in points of the media's height

Table 95: Set an Input's Status Response Attributes

For the **Set an Input's Status** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – input with specified id is not installed
422	Unprocessable Entity	Error – invalid configuration parameter. Check “Details” for more information

Table 96: Set an Input's Status HTTP Status Codes

5.2.6 Localization (Printer)

The **Localization** sub-resource contains everything needed to manage printer localization strings. This is for client applications to display localized strings for any printer info/warnings/errors messages. For more information on setting printer messages, see 5.2.2. Below is a list of actions supported under the **Localization** sub-resource and how to access it:

Action	Method	URL
Get Localization	GET	https://<fiery_ip>/fierypath/v1/system/printer/localization
Create Localization	POST	https://<fiery_ip>/fierypath/v1/system/printer/localization
Create File Localization	POST	https://<fiery_ip>/fierypath/v1/system/printer/localization/upload
Update Localization	PUT	https://<fiery_ip>/fierypath/v1/system/printer/localization
Delete Localization	DELETE	https://<fiery_ip>/fierypath/v1/system/printer/localization

Table 97: Localization Actions

The **Get Localization** action will indicate if the printer has set localization. The response body will indicate this with the content being “Localization information is available”.

For the **Get Localization** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
204	No Content	Error – no localization resource available
401	Unauthorized	Error – check for successful log-in

Table 98: Get Localization HTTP Status Codes

The **Create Localization** action creates a new localization sub-resource. The body of the HTTPS message requires a single JSON object containing an array of objects as described in the format below. This can be used for a **maximum** JSON object size of **100KB**.

Element	Type	Required	Purpose
localization	JSON Array	Yes	Contains an array of localization objects
key	JSON Object	Yes	Named message object to be used with printer status (info/warnings/errors)
lang/value	String/String	Yes	Key/value pairs defining the language code and the translation for the Message string. The language code specified can be one of the following value: "en_US", "en_GB", "fr_FR", "it_IT", "de_DE", "es_ES", "nl_NL", "pt_BR", "ja_JP", "sv_SE", "zh_CN", "da_DK", "fi_FI", "no_NO", "pl_PL", "ru_RU", "tr_TR", "cs_CZ", "zh_TW", "ko_KR"

Table 99: Create Localization

The printer does not have to provide localized strings for all the possible language codes specified above. It can send only the ones it supports and/or requires client applications to display localized messages for.

Here is an example of the JSON object for the **Create Localization** request:

```
{
    "localization": [
        {
            "This is string one": {
                "fr_FR": "C'est la première chaîne",
                "ja_JP": "これは文字列1です"
            }
        },
        {
            "This is string two": {
                "fr_FR": "C'est la chaîne deux",
                "ja_JP": "これは文字列2です"
            }
        }
    ]
}
```

For the **Create Localization** action, the valid HTTP status codes are as follows:

Code	Indication	Description
201	OK	Success
401	Unauthorized	Error – check for successful log-in
413	Payload Too Large	The request is larger than the server is willing or able to process. Previously called "Request Entity Too Large"
422	Unprocessable Entity	Error – invalid configuration parameter. Check "Details" for more information

Table 100: Create Localization HTTP Status Codes

The **Create File Localization** action is similar to the above except the same JSON object as specified above can be sent as a file to create a new localization resource. This can be used for a **maximum** file size of 5M**B**. This should be sent as "multipart/form-data". The resource will be created if and only if a single file is received.

Element	Type	Required	Purpose
file	multipart/form-data	Yes	File containing a JSON object as specified earlier

Table 101: Create File Localization

For the **Create File Localization** action, the valid HTTP status codes are as follows:

Code	Indication	Description
201	OK	Success
401	Unauthorized	Error – check for successful log-in
422	Unprocessable Entity	Error – invalid configuration parameter. Check "Details" for more information
500	Internal Server Error	Error – file maybe more than 5MB

Table 102: Create File Localization HTTP Status Codes

The **Update Localization** action updates a localization sub-resource. The body of the HTTPS message requires a single JSON object containing an array of objects as described in the format below. This action can be used to dynamically add new localization strings. This can be used for a **maximum** JSON object size of **100KB**.

Element	Type	Required	Purpose
localization	JSON Array	Yes	Contains an array of localization objects
key	JSON Object	Yes	Named message object to be used with printer status (info/warnings/errors)
lang/value	String/String	Yes	Key/value pairs defining the language code and the translation for the Message string. The language code specified can be one of the following value: "en_US", "en_GB", "fr_FR", "it_IT", "de_DE", "es_ES", "nl_NL", "pt_BR", "ja_JP", "sv_SE", "zh_CN", "da_DK", "fi_FI", "no_NO", "pl_PL", "ru_RU", "tr_TR", "cs_CZ", "zh_TW", "ko_KR"

Table 103: Update Localization

Here is an example of the JSON object for the **Update Localization** request:

```
{
    "localization": [
        {
            "This is new string three": {
                "fr_FR": "Ceci est nouvelle chaîne trois",
                "ja_JP": "これは新しい文字列3です"
            }
        },
        {
            "This is new string four": {
                "fr_FR": "Ceci est nouvelle chaîne quatre ",
                "ja_JP": "これは新しいストリング4です"
            }
        }
    ]
}
```

For the **Update Localization** action, the valid HTTP status codes are as follows:

Code	Indication	Description
201	OK	Success
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – localization resource is not created
413	Payload Too Large	The request is larger than the server is willing or able to process. Previously called "Request Entity Too Large"
422	Unprocessable Entity	Error – invalid configuration parameter. Check “Details” for more information

Table 104: Update Localization HTTP Status Codes

The **Delete Localization** action removes the localization sub-resource created earlier. An optional JSON object can be provided to remove only some localization strings. This optional JSON object can be used to remove the strings added dynamically as part of the **Update Localization** action and it needs to be in the same format. The valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – localization resource is not created
422	Unprocessable Entity	Error – invalid configuration parameter. Check “Details” for more information

Table 105: Delete Localization HTTP Status Codes

5.2.7 OEM Corrections (Printer)

The Corrections sub-resource contains everything needed to manage printer corrections -- this currently includes uniformity, nozzle-out, calibration and profiling corrections.

5.2.7.1 Nozzle-out/Uniformity

Below is a list of actions supported under the Corrections sub-resource for nozzle-out and uniformity and how to access it:

Action	Method	URL
Upload Nozzle Out Printer Corrections Files	POST	https://<fiery_ip>/fierypath/v1/system/printer/oem/corrections/nozzleOut
Set Nozzle Out Printer Corrections	PUT	https://<fiery_ip>/fierypath/v1/system/printer/oem/corrections/nozzleOut
Delete Nozzle Out Printer Corrections	DELETE	https://<fiery_ip>/fierypath/v1/system/printer/oem/corrections/nozzleOut/<id>
Delete All Nozzle Out Printer Corrections	DELETE	https://<fiery_ip>/fierypath/v1/system/printer/oem/corrections/nozzleOut
Upload Uniformity Printer Corrections Files	POST	https://<fiery_ip>/fierypath/v1/system/printer/oem/corrections/uniformity
Set Uniformity Printer Corrections	PUT	https://<fiery_ip>/fierypath/v1/system/printer/oem/corrections/uniformity
Delete Uniformity Printer Corrections	DELETE	https://<fiery_ip>/fierypath/v1/system/printer/oem/corrections/uniformity/<id>
Delete All Uniformity Printer Corrections	DELETE	https://<fiery_ip>/fierypath/v1/system/printer/oem/corrections/uniformity

Table 106: Nozzle-out/Uniformity Corrections Actions

The **Upload Nozzle Out Printer Corrections** and **Upload Uniformity Printer Corrections** actions upload one or more corrections files – which will later be referenced by the corresponding **Set Nozzle Out Printer Corrections** and **Set Uniformity Printer Corrections** actions, respectively. The body of the HTTPS messages require one or more “form-data” elements (i.e. “File1”, “File2”, …, “FileN”), each representing a file of correction data of a single colorant for a single head. For example, updating 2 heads with 4 colorants each requires uploading 8 data files.

Element	Type	Required	Purpose
file	multipart/form-data	Yes	File(s) containing nozzle out or uniformity correction data

Table 107: Upload Nozzle-Out/Uniformity Printer Correction

For the **Upload Nozzle Out Printer Corrections** and **Upload Uniformity Printer Corrections** actions, the valid HTTP status codes are as follows:

Code	Indication	Description
201	OK	Success
401	Unauthorized	Error – check for successful log-in
422	Unprocessable Entity	Error – invalid configuration parameter. Check “Details” for more information
500	Internal Server Error	Error – file maybe more than 5MB

Table 108: Upload Nozzle-Out/Uniformity Printer Correction Status Codes

If successfully uploaded, the server will respond with an HTTP status code of **201** and the body will contain an array of objects, each containing a valid **tag** as shown below:

Element	Type	Required	Purpose
File1	String	Yes	JSON Object containing File Tag element
Tag	String	Yes	Indicates the file ID for File1
File2	String	Yes	JSON Object containing the File Tag element
Tag	String	Yes	Indicates the file ID for File2
…			
FileN	String	Yes	JSON Object containing the File Tag element
Tag			Indicates the Tag ID for FileN

Table 109: Upload Nozzle Out/Uniformity Printer Correction Attributes Response Attributes

The **Set Nozzle Out Printer Corrections** and **Set Uniformity Printer Corrections** actions initiates the printer correction process. The body of the HTTPS messages require a single "corrections" JSON object, containing the several JSON elements described below:

Element	Type	Required	Purpose
manufacturer	JSON Object	Yes	JSON Object containing the attributes related to the manufacturer of the corrections device.
name	String	Yes	Indicates the name of the manufacturer.
device	String	Yes	Indicates the name of the device used to generate the correction data.
model	String	Yes	Indicates the name of the model used to generate correction data.
configuration	JSON Object	Yes	JSON Object containing the attributes related to the configuration.
format	String	Yes	Indicates the format of the correction data. Valid “format” values include: “Raw_Binary”, “Raw_Text”, “UCT_Text”, “CSV_Text” or “Raw_Tiff”. Uniformity correction data supports all “format” values except “CSV_Text”. Nozzle Out correction data supports only “CSV_Text”” value.
bitsPerPixel	Number	No	Indicates the number of bits required to represent the various drop sizes per pixel. Valid “bitsPerPixel” values include: 1, 2, 4, or 8. Required for Uniformity correction data when “format” value is “Raw_Binary” or “Raw_Text”.
numberOfLevels	Number	No	Indicates the total number of levels in the correction data. Required element when “format” value is “UCT_Text”.
numberOfNozzles	Number	Yes	Indicates the total number of nozzles.
precision	String	No	Indicates the precision of the correction data. Valid “precision” values include: “U8” or “U16”. Required for Uniformity correction data. Nozzle Out correction data supports only “U8”.
printBarSet	Array	Yes	JSON Array containing one or more entries representing a logical printbar set (e.g. front/back, different resolutions)
id	Number	Yes	Indicates the ID of the logical printbar set
side	String	No	Indicates if the printbar set is for front or back. Valid “side” values are “front”, “back” or “both. If not provided, “both” is assumed. Refer to product specific documentation to determine if this element is supported
resolution	Array	No	JSON Array containing the rip resolutions for which this print bar set is applicable. Example value - [“600x600”, “600x1200”, “1200x1200”]. If not provided or it set to [“all”], it will be applicable for all resolutions. Refer to product specific documentation to determine if this element is supported
printBars	Array	Yes	JSON Array containing one or more entries representing a physical printbar set (containing colorants/inks)
id	Number	Yes	Indicates the ID of the physical printbar set
ink	String	No	Typical values are “cyan”, “magenta”, “yellow”, “black”. The full list of supported inks is product specific. Refer to product specific documentation to determine if this element is supported
file	JSON Object	Yes	JSON Object containing the attributes related to the file.
tag	String	Yes	File id returned by Upload Nozzle Out/Uniformity Printer Correction Files

Table 110: Corrections Attributes

For **Set Nozzle Out Printer Corrections** and **Set Uniformity Printer Corrections** actions, the valid HTTP status codes are as follows:

Code	Indication	Description
202	OK	Accepted
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – input with specified id is not installed
422	Unprocessable Entity	Error – invalid configuration parameter. Check “Details” for more information

Table 111: Set Nozzle Out/Uniformity Printer Correction Attributes HTTP Status Codes

Response code 202 indicates that DFE has accepted correction data for processing. It does not mean correction data has been processed. DFE will send **dfeUniformityCorrectionsUpdated** / **dfeNozzleOutCorrectionsUpdated** event once it finishes processing the corrections data. The event has {“successful”: true} if processing of correction data is successfully applied, {“successful”: false, “error”: “<error message>”} in case of an error.

Here is a snippet of printBarSet corrections attributes:

```
"printBarSet": [{
    "id": 0,    → ID for the logical printBar set, e.g. this could represent the set for one of the supported resolutions
    "printBars": [{
        "id": 0,    → ID for the physical printBar e.g. this could be for cyan
        "file": {
            "tag": "e9f685725e3ecccfaf21fe489c910486"
        }
    }, {
        "id": 1,
        "file": {
            "tag": "925ee72b2a87e8e7a0a8238537b86296"
        }
    },
```

The **Delete Nozzle Out Printer Corrections** and **Delete Uniformity Printer Corrections** actions removes the oem corrections of given printBarSet ID or all printBarSet ID’s if an ID is not specified. The valid HTTP status codes are as follows:

Code	Indication	Description
202	OK	Accepted
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – oem configuration resource is not created

Table 112: Delete Nozzle Out/Uniformity Printer Correction Attributes HTTP Status Codes

Response code 202 indicates that DFE has accepted the request to clear the correction data. It does not mean the correction setting has been cleared. DFE will send **dfeUniformityCorrectionsCleared** / **dfeNozzleOutCorrectionsCleared** event once

it finishes clearing of corrections setting. The event has {“successful”: true} for successful operation {“successful”: false} in case of an error.

5.2.7.2 Calibration/Profiling

Below is a list of actions supported under the Corrections sub-resource for calibration and profiling and how to access it:

Action	Method	URL
Set Calibration Status	PUT	https://<fiery_ip>/fierypath/v1/system/printer/oem/corrections/calibration/status
Upload Calibration Printer Corrections File	POST	https://<fiery_ip>/fierypath/v1/system/printer/oem/corrections/calibration/upload
Set Profiling Status	PUT	https://<fiery_ip>/fierypath/v1/system/printer/oem/corrections/profiling/status
Upload Profiling Printer Corrections File	POST	https://<fiery_ip>/fierypath/v1/system/printer/oem/corrections/profiling/upload

Table 113: Calibration/Profiling Corrections Actions

The **Set Calibration Status** and **Set Profiling Status** actions allows setting the status for a calibration or profiling measurement job. The body of the HTTPS message requires a single JSON object, containing the JSON element described below.

Element	Type	Required	Purpose
action	String	Yes	Valid values include: “ready”, “resume”, “waiting”, “measuring”, “processing”, “error” or “offline

Table 114: Set Calibration/Profiling Status Request Attributes

For the **Set Calibration Status** and **Set Profiling Status** actions, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
422	Unprocessable Entity	Error – invalid configuration parameter. Check “Details” for more information

Table 115: Set Calibration/Profiling Status HTTP Status Codes

The **Upload Calibration Printer Corrections File** and **Upload Profiling Printer Corrections File** actions allows the printer to upload calibration or profiling corrections file. This can be used for a **maximum** file size of 25M**B**. This should be sent as “multipart/form-data”. The resource will be created if and only if a single file is received.

Element	Type	Required	Purpose
file	multipart/form-data	Yes	File containing calibration/profiling corrections data

Table 116: Upload Calibration/Profiling Corrections FIle

For the above actions, the valid HTTP status codes are as follows:

Code	Indication	Description
201	OK	Success
401	Unauthorized	Error – check for successful log-in
422	Unprocessable Entity	Error – invalid configuration parameter. Check “Details” for more information
500	Internal Server Error	Error – file maybe more than 25MB

Table 117: Upload Calibration/Profiling Corrections File HTTP Status Codes

5.2.8 Finishers (Printer)

The **Finishers** sub-resource contains everything needed to manage the list of finishers. Below is a list of actions supported under the **Finishers** sub-resource and how to access it:

Action	Method	URL
Get a List of Installed Finishers	GET	https://<fiery_ip>/fierypath/v1/system/printer/finishers
Create a Finisher	POST	https://<fiery_ip>/fierypath/v1/system/printer/finishers
Delete a Finisher	DELETE	https://<fiery_ip>/fierypath/v1/system/printer/finishers/<id>
Delete all Finishers	DELETE	https://<fiery_ip>/fierypath/v1/system/printer/finishers

Table 118: Finisher Actions

The **Get a List of Installed Finishers** action retrieves the current list of finisher entries. The response body will contain a JSON array (based on the number of installed finishers), containing the JSON elements described below:

Element	Type	Required	Purpose
id	Number	Yes	Indicates the ID of the finisher
type	String	Yes	Valid values are printer specific
value	String	No	Valid values are printer specific

Table 119: Get a List of Installed Finishers Response Attributes

If successful, **id** will be a number (which is required for accessing other resources related to this finisher) valid only for the duration of the session.

For the **Get a List of Installed Finishers** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
204	No Content	Error – no installed finisher
401	Unauthorized	Error – check for successful log-in

Table 120: Get a List of Installed Finishers HTTP Status Codes

The **Create a Finisher** action creates a new finisher and adds it to the list of installed finishers. The body of the HTTPS message requires a single **finisher** JSON object, containing the JSON element described below:

Element	Type	Required	Purpose
type	String	Yes	Indicates the type of finisher installed. Valid values are printer specific
value	String	No	Valid values are printer specific

Table 121: Create a Finisher Request Attributes

Here is an example of the JSON object for the **Create a Finisher** request:

```
{
    "finisher": {"type": "printer_specific"}
}
```

For the **Create a Finisher** action, the body of the response contains the JSON element described below:

Element	Type	Required	Purpose
id	Number	Yes	Indicates the ID of the finisher

Table 122: Create a Finisher Response Attributes

For the **Create a Finisher** action, the valid HTTP status codes are as follows:

Code	Indication	Description
201	OK	Success
401	Unauthorized	Error – check for successful log-in
422	Unprocessable Entity	Error – invalid configuration parameter. Check “Details” for more information

Table 123: Create a Finisher HTTP Status Codes

The **Delete a Finisher** action removes the given finisher from the list of installed finishers. The valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – finisher with specified id is not installed

Table 124: Delete a Finisher HTTP Status Codes

The **Delete all Finishers** action removes all the installed finishers. The valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – no installed finishers

Table 125: Delete all Finishers HTTP Status Codes

5.2.9 Outputs (Printer)

The **Outputs** sub-resource contains everything needed to manage the list of outputs. Below is a list of actions supported under the **Outputs** sub-resource and how to access it:

Action	Method	URL
Get a List of Installed Outputs	GET	https://<fiery_ip>/fierypath/v1/system/printer/outputs
Create an Output	POST	https://<fiery_ip>/fierypath/v1/system/printer/ outputs
Delete an Output	DELETE	https://<fiery_ip>/fierypath/v1/system/printer/ outputs/<id>
Delete all Outputs	DELETE	https://<fiery_ip>/fierypath/v1/system/printer/ outputs

Table 126: Output Actions

The **Get a List of Installed Outputs** action retrieves the current list of output entries. The response body will contain a JSON array (based on the number of installed outputs), containing the JSON elements described below:

Element	Type	Required	Purpose
id	Number	Yes	Indicates the ID of the output
type	String	Yes	Valid values are printer specific
value	String	No	Valid values are printer specific

Table 127: Get a List of Installed Outputs Response Attributes

If successful, **id** will be a number (which is required for accessing other resources related to this output) valid only for the duration of the session.

For the **Get a List of Installed Outputs** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
204	No Content	Error – no installed output
401	Unauthorized	Error – check for successful log-in

Table 128: Get a List of Installed Outputs HTTP Status Codes

The **Create an Output** action creates a new output and adds it to the list of installed outputs. The body of the HTTPS message requires a single **output** JSON object, containing the JSON element described below:

Element	Type	Required	Purpose
type	String	Yes	Indicates the type of output installed. Valid values are printer specific
value	String	No	Valid values are printer specific

Table 129: Create an Output Request Attributes

Here is an example of the JSON object for the **Create an Output** request:

```
{
    "output": {"type": "printer_specific"}
}
```

For the **Create an Output** action, the body of the response contains the JSON element described below:

Element	Type	Required	Purpose
id	Number	Yes	Indicates the ID of the output

Table 130: Create an Output Response Attributes

For the **Create an Output** action, the valid HTTP status codes are as follows:

Code	Indication	Description
201	OK	Success
401	Unauthorized	Error – check for successful log-in
422	Unprocessable Entity	Error – invalid configuration parameter. Check “Details” for more information

Table 131: Create an Output HTTP Status Codes

The **Delete an Output** action removes the given output from the list of installed outputs. The valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – output with specified id is not installed

Table 132: Delete an Output HTTP Status Codes

The **Delete all Outputs** action removes all the installed outputs. The valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – no installed output

Table 133: Delete all Outputs HTTP Status Codes

5.2.10 Info (DFE)

The **Info** sub-resource contains everything needed to get DFE information. Below is a list of actions supported under the info sub-resource and how to access it:

Action	Method	URL
Get DFE Info	GET	https://<fiery_ip>/fierypath/v1/system/dfe/info

Table 134: DFE Info Action

The **Get DFE Info** action retrieves the current DFE information entry. The body of the response contains the JSON element described below:

Element	Type	Required	Purpose
version	Number	Yes	Indicates the current DFE/Fiery Path version.
settings	JSON Object	No	Contains printer specific settings

Table 135: Get DFE Info Response Attributes

For the **Get DFE Info** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – this action is not supported

Table 136: Get DFE Info HTTP Status Codes

5.3 Jobs

The **Jobs** resource contains all the sub-resources that handle job related actions.

5.3.1 Jobs

The **Jobs** sub-resource contains everything needed to manage the printing of jobs. Below is a list of actions supported under the **Jobs** sub-resource and how to access it:

Action	Method	URL
Set a Job's Status	PUT	https://<fiery_ip>/fierypath/v1/jobs/<id>/status

Table 137: Job Actions

The **Set a Job's Status** action updates the current status for the given job. This action is used by a **Printer Controller** client as the response to the several Job and Image events (described in earlier sections). The body of the HTTPS message requires a single **status** JSON object, containing the JSON element described below:

Element	Type	Required	Purpose
state	String	Yes	Indicates the state of the action. Can be "start", "complete", or "cancel"
image	JSON Object	No	JSON Object containing the attributes related to the image.
id	String	No	ID of the image
offset	Number	No	Indicates the offset (in pixels), the image should be shifted when transferred to the printer. If the image is part is part of a group, the offset specified should be the same for all the images in that group. This is only valid in response to an imageStarting event
display	Boolean	No	Indicates the image should be marked as printed for display purposes only. This is only valid in response to an imageCompleting event. This can be used in cases when the printer does not want to send the "complete" indicating the image is printed but still wants the display to be updated to show progress. It is necessary for the printer to issue a second response for imageCompleting with a "complete" without setting this attribute when it is ready to notify the DFE that the image is printed
printable	Boolean	No	Indicates whether the image should be printed. This can be used in cases when the printer does not want to "print" the given image, but still print other images of the job. If not supplied, the DFE will assume the image is "printable". This is only valid in response to an imageStarting event
ink	JSON Array	No	Optional JSON Array indicating total ink used per id. This is only valid in response to an jobCompleting or jobCanceling event. It is not valid when the "image" element is specified. Each element of the array should have an "id" of the ink, followed by the total amount under "used"
id	Number	No	ID of the ink
used	Number	No	Indicates the total amount of ink used for the job in milliliters

Table 138: Set a Job's Status Request Attributes

When responding to an **imageStarting** or **imageCompleting** event, the **image** attribute is required. The **offset** attribute is optional but is only applicable when responding to an **imageStarting** event. Here are examples of some events and possible status objects for their corresponding **Set a Job's Status** action:

event: **jobStarting** {"status": {"state": "start"}}
event: **imageStarting** {"status": {"state": "start", "image": {"id": "1", "offset": 0}}}
event: **imageCompleting** {"status": {"state": "complete", "image": {"id": "1"}}}
event: **jobCompleting** {"status": {"state": "complete"}}

Note that the **jobCompleting** or **jobCanceling** response should only be sent after the last **imageCompleting** response. Otherwise, job accounting information will be inaccurate. After sending a **jobCanceling** response, all outstanding **imageCompleting** events will be assumed by the DFE to have not printed and will be cleaned-up, accordingly.

If a **jobCanceling** event is received, the printer should respond as follows to the DFE events:

```
{
    "status": {"state": "cancel"}
}
```

In the case of **jobCompleting** or **jobCanceling**, an optional "**ink**" array can be specified indicating the total ink used per ink "**id**". This amount needs to be specified in **milliliters**. An example would be:

event: **jobCompleting** {"status": {"state": "complete", "ink": [{"id": 1, "used": 0.445}, [{"id": 2, "used": 0.151}]}}
event: **jobCanceling** {"status": {"state": "cancel", "ink": [{"id": 1, "used": 0.235}, [{"id": 2, "used": 0.113}]}}

For the **Set a Job's Status** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – id or action not found
422	Unprocessable Entity	Error – invalid configuration parameter. Check "Details" for more information

Table 139: Set a Job's Status HTTP Status Codes

For all Job Status actions when successful, in addition to receiving an HTTP status code of 200, the response will contain a JSON object with one element – { success : true }. In general, the printer should retry on receiving an error (HTTP status code 401, 422, 503 or any other code) except when it receives HTTP status code 404 which indicates that the DFE received the request but cannot complete it.

5.3.2 Scan Jobs

The **Scan Jobs** sub-resource contains everything needed to manage the scanning of jobs. Below is a list of actions supported under the **Scan Jobs** sub-resource and how to access it:

Action	Method	URL
Initiate a Scan Job	POST	https://<fiery_ip>/fierypath/v1/scan/initiate
Set a Scan Job's Status	PUT	https://<fiery_ip>/fierypath/v1/scan/<id>/status

Table 140: Scan Job Actions

The **Initiate a Scan Job** action allows the printer to initiate a scan job. This action is optional and is only needed if the Printer Controller client wants to initiate a scan job. The body of the HTTPS message requires a single **settings** JSON object, containing the JSON element described below:

Element	Type	Required	Purpose
settings	JSON Object	Yes	JSON Object containing scan specific attributes. Refer to product specific documentation for details

Table 141: Initiate a Scan Job Request Attributes

For the **Initiate a Scan Job** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
403	Forbidden	Error – Check “Message” for more information
422	Unprocessable Entity	Error – invalid configuration parameter. Check “Details” for more information

Table 142: Set a Scan Job's Status HTTP Status Codes

The **Set a Scan Job’s Status** action updates the current status for the given job. This action is used by a **Printer Controller** client as the response to events from DFE and to initiate scan specific actions. The body of the HTTPS message requires a single **status** JSON object, containing the JSON element described below:

Element	Type	Required	Purpose
state	String	Yes	Indicates the state of the action. Can be "start", “create”, "complete", “cancel” or “abort”
settings	JSON Object	No	JSON Object containing scan specific attributes. This is required only when using “create” action. Refer to product specific documentation for details
filepath	String	No	Indicates the fully qualified path to the image. This is required only when using “create” action
reason	String	No	Indicates the reason for aborting the scan job. This is optional and applicable only when using “abort” action

Table 143: Set a Scan Job's Status Request Attributes

Here are examples of actions initiated by the printer and the status objects for its corresponding **Set a Job’s Status** action:

start: {“status”: {“state”: "start"}}
create: {“status”: {“state”: "create”, “settings”: {“key1”: “value1”}, “filepath”: “C:\image\test.jpg”}}
complete: {“status”: {“state”: "complete"}}
cancel: {“status”: {“state”: "cancel"}}
abort: {“status”: {“state”: "abort", “reason”: “specify reason for abort”}}

Note that the “complete” action should only be initiated once the printer has added all the pages using “create” action. The “cancel” action should be a response to the **scanJobCanceling** event. If the printer wants to initiate a cancel for an active scan, it can send the “abort” action. If a reason needs to be specified for cancelation, that can be done using the optional reason attribute.

For the **Set a Scan Job’s Status** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
404	Not Found	Error – id or action not found
422	Unprocessable Entity	Error – invalid configuration parameter. Check “Details” for more information

Table 144: Set a Scan Job's Status HTTP Status Codes

For all Scan Job Status actions when successful, in addition to receiving an HTTP status code of 200, the response will contain a JSON object with one element – { success : true }. In general, the printer should retry on receiving an error (HTTP status code 401, 422, 503 or any other code) except when it receives HTTP status code 404 which indicates that the DFE received the request but cannot complete it.

5.3.3 Maintenance Jobs

The **Maintenance** sub-resource contains everything needed to manage the setup and insertion of maintenance jobs. Below is a list of actions supported under the **Maintenance Jobs** sub-resource and how to access it:

Action	Method	URL
Setup Maintenance Jobs	POST	https://<fiery_ip>/fierypath/v1/maintenance/settings
Delete Maintenance Jobs	DELETE	https://<fiery_ip>/fierypath/v1/maintenance/settings
Insert Maintenance Job	PUT	https://<fiery_ip>/fierypath/v1/maintenance/<name>/status

Table 145: Maintenance Job Actions

The **Setup Maintenance Jobs** action by the printer sends the list of maintenance jobs to the DFE. The body of the HTTPS message requires a single **status** JSON object, containing the JSON elements described below:

Element	Type	Required	Purpose
settings	JSON Object	Yes	JSON Object containing maintenance joblist.
joblist	JSON Array	Yes	JSON Array indicating the maintenance jobs. Each element of the array should have a “name” of the job and optionally a “mode” attribute. The maximum size of this array can be 3. If the mode is not specified, it will default to “immediate”
name	String	Yes	Indicates the name of the maintenance job
mode	String	No	Indicates the mode. The supported values are “immediate” and “jobend”. The “immediate” mode allows the client to insert the maintenance job on demand. If the “jobend” mode is used, the DFE will insert the maintenance job at the end of every job that is printed

Table 146: Setup Maintenance Jobs Attributes

For the **Setup Maintenance Jobs** action, the valid HTTP status codes are as follows:

Code	Indication	Description
201	OK	Success
401	Unauthorized	Error – check for successful log-in
422	Unprocessable Entity	Error – invalid configuration parameter. Check “Details” for more information

Table 147: Setup Maintenance Jobs HTTP Status Codes

The **Delete Maintenance Jobs** action removes all the maintenance jobs previously specified. The valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
422	Unprocessable Entity	Error – invalid configuration parameter. Check “Details” for more information

Table 148: Delete Maintenance Jobs HTTP Status Codes

The **Insert Maintenance Job** action inserts the previously specified maintenance job if the “start” action is specified. This job must have been printed once. If the mode specified for this job earlier is “immediate”, the DFE will insert the maintenance job as soon as it can. If the mode is “jobend”, the DFE will start inserting this job at the end of every job that is printed. The body of the HTTPS message requires a single **status** JSON object, containing the JSON element described below:

Element	Type	Required	Purpose
state	String	Yes	Indicates the state of the action. Can be "start" or “stop”.

Table 149: Insert Maintenance Job Attributes

Here is an example of the status object:

start**:** {“status”: {“state”: "start"}}

Note that a maintenance job cannot be inserted when Fiery is idle. Also, it is necessary to print this maintenance job once at startup. Additionally, if a request has been sent by the printer to insert a maintenance job that has been setup for “immediate” mode, another request will succeed only once the current maintenance job is inserted and printed.

The ”stop” action will let the DFE know to not insert the specified maintenance job. This action is more suitable for jobs that have the “jobend” mode specified.

For the **Insert Maintenance Job** action, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
401	Unauthorized	Error – check for successful log-in
422	Unprocessable Entity	Error – invalid configuration parameter. Check “Details” for more information

Table 150: Insert Maintenance Job HTTP Status Codes

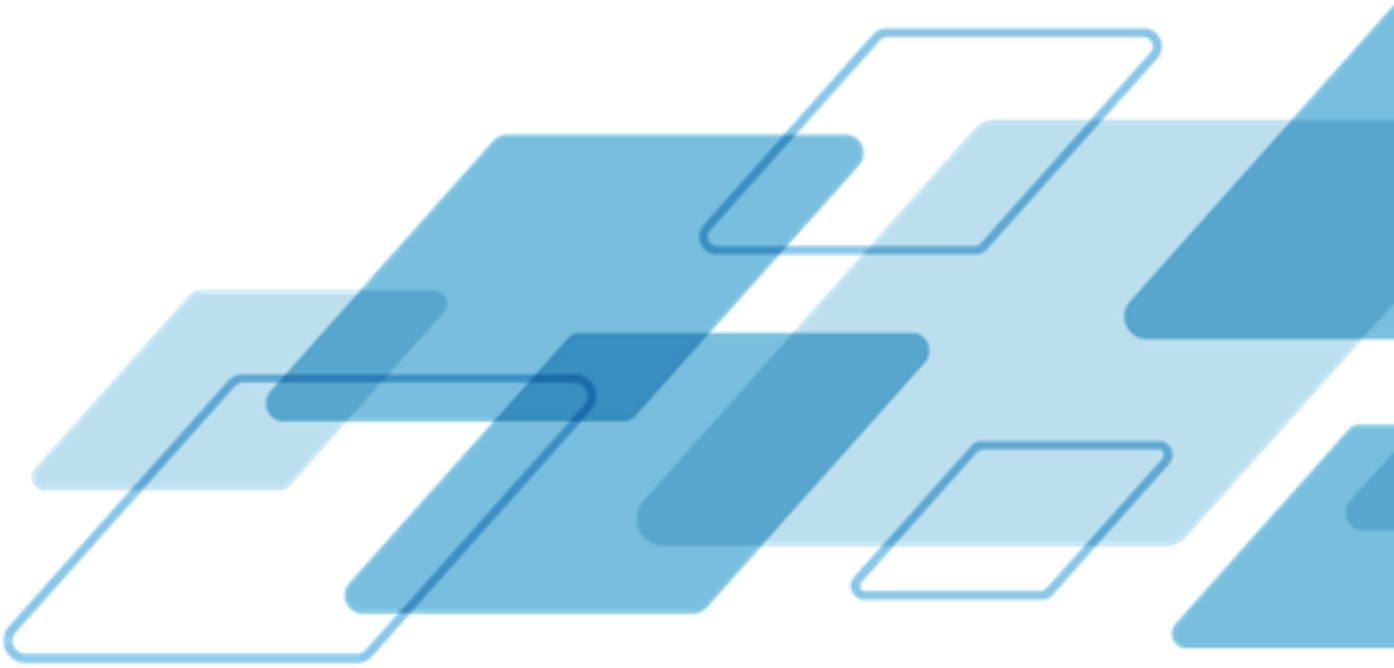
For the Insert Maintenance Job action when successful, in addition to receiving an HTTP status code of 200, the response will contain a JSON object with one element – { success : true }.

5.4 HTTP Status Codes

The **FieryPath API** uses standard HTTP status codes to communicate success or failure of an API. In general, a successful API will return either status code **200** or **201**. All other status codes indicate an error. For the **FieryPath API**, the valid HTTP status codes are as follows:

Code	Indication	Description
200	OK	Success
201	Created	The request has been fulfilled, resulting in the creation of a new resource
202	Accepted	The request has been accepted for processing, but the processing has not been completed
204	No Content	The server successfully processed the request and is not returning any content
401	Unauthorized	Check for successful log-in
403	Forbidden	The request was a valid request, but the server is refusing to respond to it
404	Not Found	The requested resource could not be found but may be available in the future
412	Precondition failed	A required precondition failed
413	Payload Too Large	The request is larger than the server is willing or able to process. Previously called "Request Entity Too Large”
422	Unprocessable Entity	The request was well-formed but was unable to be followed due to semantic or parameter errors. Check “Details” for more information
429	Too many requests	Operation already in progress
500	Internal Server Error	A generic error message, given when an unexpected condition was encountered, and no more specific message is suitable
503	Service Unavailable	The server is not ready to handle the request at this time. Try again later

Table 151: HTTP Status Codes



For more information, please visit fiery.com.

Nothing herein should be construed as a warranty in addition to the express warranty statement provided with Fiery, LLC products and services.

ColorGuard, ColorRight, Command WorkStation, ColorWise, Fiery, the Fiery logo, Fiery Compose, Fiery Driven, the Fiery Driven logo, Fiery Edge, Fiery Essential, Fiery HyperRIP, Fiery Impose, Fiery Impress, Fiery ImageViewer, Fiery Intensify, Fiery JobExpert, Fiery JobFlow, Fiery JobMaster, Fiery Prep-it, Fiery Prints, the Fiery Prints logo, Fiery TrueBrand, FreeForm, MicroPress, IQ, PrintMe, RIPChips, RIP-While-Print, Spot-On, Spot Pro, and WebTools are trademarks or registered trademarks of Fiery, LLC and/or its wholly owned subsidiaries in the U.S. and/or certain other countries. All other terms and product names may be trademarks or registered trademarks of their respective owners and are hereby acknowledged.

© 2023 FIERY, LLC. ALL RIGHTS RESERVED.

FTL_XXX.XX.23